F24-014-D-Sehat-e-Tafseel

Project Team

Azmeer Sohail Khan 21I-2977
Hania Zahid 22I-1270
Laiba Shafiq 21I-0840

Session 2021-2025

Supervised by

Mr. Muhammad Owais Idrees

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The purpose of this project proposal is to address several critical inefficiencies in Healthcare Systems of Pakistan and these are being addressed by creating a Patient Record Database. The current hospitals work in silos, where each department manages itself and has little to no integration with others leading to repetitive history taking, repeating of tests (sometimes even within the same organization) and un-standardized patient care. Many patients misplace their medical records, existing systems do not allow collaboration between hospitals or elevate cutting-edge tools such as AI chatbots. Not only that, a lot of patients have told me how they could never get an accurate diagnosis or find the right specialists as dermatological care needs to be improved.

To solve them, this system gave a unique reference ID to every patient which is associated with their CNIC and the complete history of that one single ID could be accessed on any hospital network. In the system, there will be a healthbot.. Patients will also be supported by a healthbot which is an AI-powered system, and can help in regards to skin diseases for dermatological patients through disease identification along with recommendations of doctors.

1.1 Problem Statement

Pakistan's healthcare system is facing a critical challenge.They do not have a centralized database, where records of patients are stored. This, in turn can cause inefficiencies like same history taken again and again from the patient working up all over somewhere else or duplicated tests being carried out on a person, healthcare following no straight line etc. There the medical records for a long time for some patients get lost, and it causes extra trouble as their charts are not online yet. This is not only a waste of a patient's precious time and resources but also increases the risk of errors in diagnosis and treatment. Existing systems are majorly manual or very limited functionalities. These systems do not facilitate interactive collaboration and good connection between different hospitals.They only maintain records of patient's of their own hospital. Also existing systems do not have medical chatbots. One big issue in Pakistan is skincare. Patients hesitate to tell their skin diseases to each other and don't find out which disease they have and don't find out which doctor suits their skin disease and how to diagnose their disease. So the main issue is clear, without a single system healthcare providers work separately and the one who suffers are patients.

1.2 Scope

Our project, Sehat-e-Tafseel is a comprehensive healthcare web application designed to connect doctors and patients, through a centralized platform. The objective is to enhance patient experience by providing automated solutions such as healthbot, which will analyze symptoms, predict the disease and recommend doctors for consultation. Additionally, patients can also book appointments. Our project also allows skin disease detection through image uploads. Patient data will be securely stored in a centralized database, ensuring easy access for the doctors. The patients will be assigned a unique ID based on their CNIC number, which can be used across multiple hospitals. The admin interface manages the hospital data, user accounts and the overall system maintenance. The scope includes patient data storage, booking consultations, and disease prediction, making it an essential tool for hospitals.

1.3 Modules

The system interfaces have been classified into six modules. These include the admin, patient, doctor, centralized database, healthbot and skin disease detection module.

1.3.1 Admin Module

This module is designed to manage the patients and hospitals data.

1. Assign and manage roles
2. Manage patient record and maintain security
3. Generate slips and assign unique ID
4. Manage patient forms (application forms, operation forms, discharge forms, consent forms)
5. Data analysis and prediction

1.3.2 Patient Module

This module provides the interface for patients to view their medical record (including diagnoses, treatments, prescriptions, test reports, surgeries, medications and previous hospitals visited) and interact with the doctors through the following features:

1. View medical record
2. Healthbot
3. Search doctor by speciality
4. View health blogs
5. Appointment booking
6. Skin disease detection

1.3.3 Doctor Module

This module provides the interface for doctors to interact with the patients and view their medical record for better treatment.

1. View assigned patients and their medical record
2. View symptoms submitted by patients
3. Update patient records
4. Add remarks
5. Generate prescription

1.3.4 Centralized Database Module

This module stores all the patient's complete medical records including diagnoses, treatments, prescriptions, test reports, surgeries, medications and previous hospitals visited, in a centralized database.

1. Store patient data
2. Hospitals access to patient data
3. Privacy and security of the database
4. Real-time access and update.

1.3.5 Healthbot

This module provides AI assistance to enhance a patient's experience by offering health assistance. It bridges the gap between patients and doctors.

1. Enter symptoms
2. Identification of possible diseases
3. Doctor Recommendation

1.3.6 Skin Disease Detection

This module is designed to help patients in the identification of possible skin conditions through image analysis. Computer vision techniques combined with machine learning algorithms will help detect potential skin diseases/allergies based upon the images uploaded.

1. Upload image
2. Identification of possible allergies/diseases
3. Doctor recommendation

1.4 User Classes and Characteristics

Table 1.1 User Classes and Characteristics

User Classes	Description
Patient	Patients that are individuals seeking medical assistance. They can interact with the system by using the healthbot, uploading pictures to identify skin diseases, retrieving their medical history and by booking appointments. The patients can use the system on their computers, laptops and mobile phones. It is anticipated that around 70 % of the patients will use the system on their mobile phones.
Doctor	Doctors will use the system to offer healthcare services. The system allows the doctors to schedule their appointments, access patient medical records, look over the test findings and interact with the patients.
Admin	Admin is in charge of managing the system's operations, maintaining patient data security and privacy and also managing the patient/doctor accounts. They can also handle technical problems and can generate reports from the centralized database.

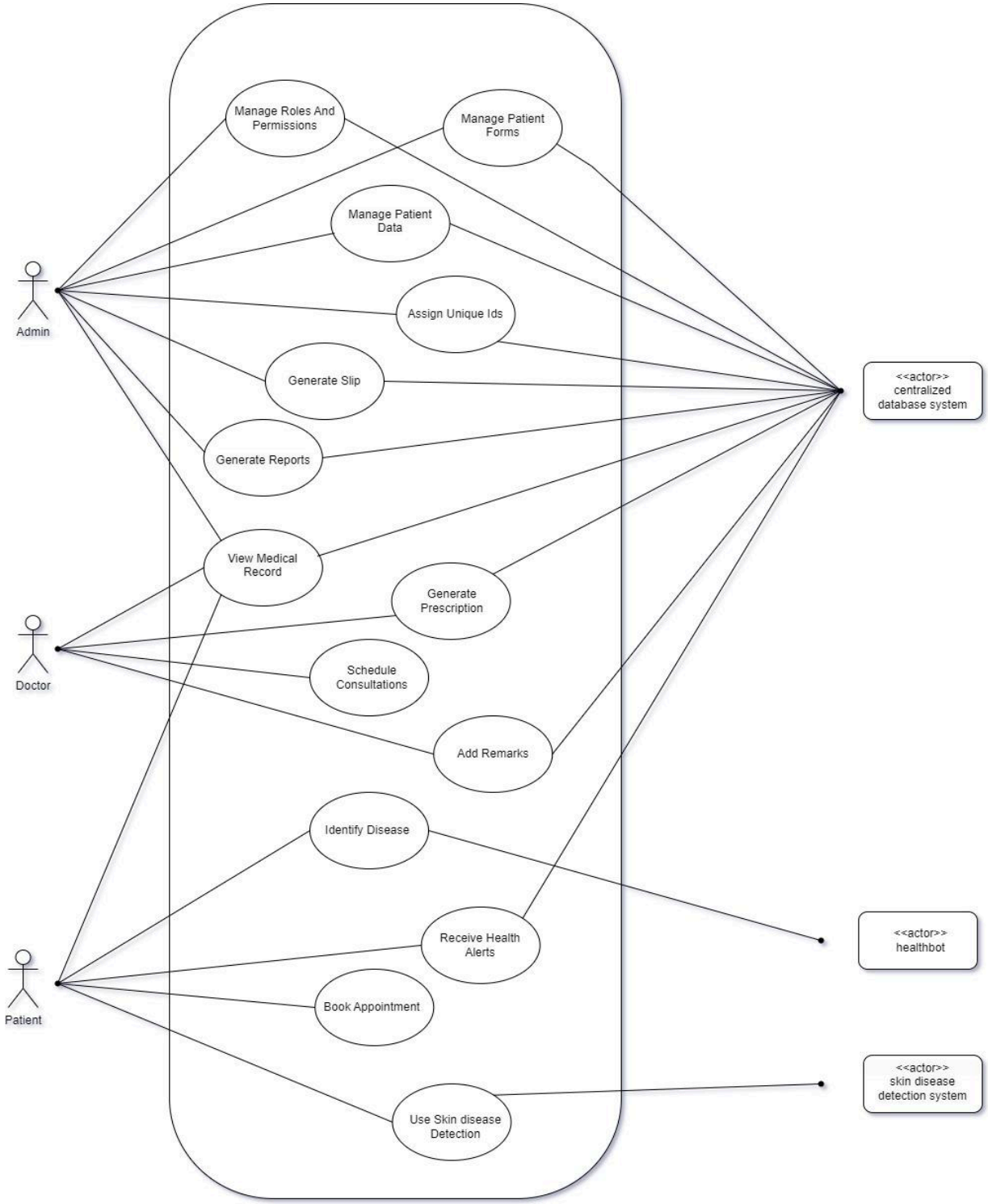
Chapter 2

Project Requirements

2.1 Use-case

2.1.1 Use-case Diagram

Fig 2.1 Use-case Diagram



2.1.2 High-level Use-cases

1. Manage Roles and Permissions

Use-case	Manage roles and permissions
Actors	Admin
Type	Primary
Description	The admin manages the roles for different users (patient and doctors). It can assign, remove or update permissions based on user requirements.

Table 2.1.1 Manage Roles and Permissions

2. Manage Patient Data

Use-case	Manage patient data
Actors	Admin
Type	Primary
Description	The admin manages the patient data including searching patient records, updating, adding and deleting them.

Table 2.1.2 Manage Patient Data

3. Assign Unique IDs

Use-case	Assign unique IDs
Actors	Admin
Type	Primary
Description	The admin will assign a unique ID based on the user's CNIC or passport number during registration.

Table 2.1.3 Assign Unique IDs

4. Generate Reports

Use-case	Generate reports
Actors	Admin
Type	Primary
Description	The admin can generate several types of reports based on specific criteria.

Table 2.1.4 Generate Reports

5. Manage Forms

Use-case	Manage forms
Actors	Admin
Type	Primary
Description	The admin manages various patient forms including the operation, consent, discharge and emergency forms.

Table 2.1.5 Manage Forms

6. Generate Slips

Use-case	Generate slips
Actors	Admin
Type	Primary
Description	Admin generates slips for in-person appointment bookings.

Table 2.1.6 Generate Slips

7. View Medical Record

Use-case	View medical record
Actors	Admin, Doctor, Patient
Type	Primary
Description	The user can view the medical records based on their respective roles.

Table 2.1.7 View Medical Record

8. Generate Prescriptions

Use-case	Generate prescriptions
Actors	Doctor
Type	Primary
Description	Doctor generates a prescription for a patient which is stored in the centralized database.

Table 2.1.8 Generate Prescriptions

9. Schedule Consultations

Use-case	Schedule consultations
Actors	Doctor
Type	Primary
Description	The doctors schedule in-person appointment bookings and can add free slots for booking.

Table 2.1.9 Generate Reports

10. Add Remarks

Use-case	Add remarks
Actors	Doctor
Type	Primary
Description	The doctor can search for the patient's medical history and can add remarks on previous treatments.

Table 2.1.10 Add Remarks

11. Identify Disease

Use-case	Identify disease
Actors	Patient
Type	Primary
Description	Patient uses the healthbot to identify the disease by giving symptoms as input and it then recommends the relevant doctors

Table 2.1.11 Identify Disease

12. Book Appointment

Use-case	Book appointment
Actors	Patient
Type	Primary
Description	The patient searches for a doctor and books an appointment at a suitable time.

Table 2.1.12 Book Appointment

13. Receive health alerts

Use-case	Receive health alerts
Actors	Patient
Type	Primary
Description	Patient receives the health alerts generated by the system and takes the necessary actions.

Table 2.1.13 Receive health alerts

14. Skin Disease Detection

Use-case	Skin disease detection
Actors	Patient
Type	Primary
Description	Patients upload their skin picture to identify the disease and the system then recommends the relevant doctors.

Table 2.1.14 Skin Disease Detection

2.1.3 Extended Use-cases

1. **Use-case name:** Manage Roles and Permissions

Scope: Sehat-e-Tafseel

Level: User-goal

Primary actor: Admin

Secondary actor: System (Centralized Database)

Stakeholders and interest:

i. **Admin**: interested in managing roles and permissions

Pre-condition: Admin has logged in and has privileges required to manage the roles.

Post-condition: Roles are assigned and updated in the database.

Main success scenario:

Actor	System
1. Admin logs in	
2. Admin navigates to the “Manage Roles and Permissions” section	
3. Admin then selects the user	4. System displays the current permissions assigned with the selected user
5. Admin add/removes the permissions	
6. Admin reviews and confirms the changes	7. System updates the permissions in the database and sends a confirmation notification to the Admin
	8. System ensures that the changes made are enforced in the user sessions

Extensions:

i. **Permission conflict:** If the admin assigns conflicting permissions then the system will prompt a warning message

ii. **Unauthorized access:** If the admin access roles or permissions that are not assigned to him/her , the system will display an error message

2. **Use-case name:** Manage Patient Data

Scope: Sehat-e-Tafseel

Level: User-goal

Primary actor: Admin

Secondary actor: System (Centralized Database)

Stakeholders and interest:

i. **Admin:** interested in managing the patient data

Pre-condition: Admin has logged in and has privileges required to manage the patient data.

Post-condition: Any modifications in the patient data are updated in the centralized database

Main success scenario:

Actor	System
1. Admin logs in	
2. Admin navigates to the 'Manage Pateint data' section	
3. Admin searches for a specific patient	4. System retrieves and displays the selected patients data
5. Admin updates the patients data (add new patient record, delete record)	6. System updates the data in the database
7. Admin logs out	

Extensions:

i. **Patient record not found:** If the searched patient id is not found, the system will display an error message saying "No patient with this ID exists"

ii. **Incorrect data:** If the admin enters invalid or incomplete data, the system will prompt an error message saying "Enter correct information"

3. **Use-case name:** Assign Unique IDs

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: System

Secondary actor: Admin

Stakeholders and interest:

i. **Admin:** interested in assigning a unique ID based on CNIC or passport number

Pre-condition: Admin has logged in and has the CNIC or passport number of the patient

Post-condition: Each patient has a unique ID based on their CNIC or passport number

Main success scenario:

Actor	System
1. Admin logs in	
2. Admin navigates to the 'Add Patient' section	3. System displays a registration form
4. Admin enters the CNIC or passport number	5. System checks if the provided CNIC or passport number is in the correct format
	6. System generates a unique ID and associates it with the new patient
7. Admin completes the registration form and submits it	8. System saves the new patient record
	9. System will display a confirmation message saying "Patient has been registered successfully"

10. Admin views the patient record for verification	

Extensions:

i. **Invalid Identifier:** if the format of CNIC or passport number is not valid, the system will display an error message saying “Invalid format”

ii. **Incomplete Information:** If any of the required fields are not filled before registration, the system will display an error message saying “Fill all the required fields”

iii. **Technical Issue:** If the system fails to assign a unique ID or register the patient due to a technical issue, the system will display a message saying “Technical Issue: Retry the Registration Process”

4. **Use-case name:** Generate Reports

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Admin

Secondary actor: System (Centralized Database)

Stakeholders and interest:

i. **Admin:** interested in generating various types of reports

Pre-condition: Admin has logged and has privileges required to generate reports.

Post-condition: Reports are generated and can be viewed or downloaded.

Main success scenario:

Actor	System
1. Admin logs in	

2. Admin navigates to the ‘Generate Reports’ section	3. System redirects to the generate reports section
4. Admin selects a specific type of report	5. System then displays the report criteria
6. Admin clicks the ‘Generate Report’ button	7. System generates the report in the correct format
8. Admin can select the ‘View Report’ or ‘Download Report’ button	

Extensions:

i. **No data available:** If there is no data for the selected criteria, the system will display a message saying “No data matches the selected criteria”

ii. **Report Generation Failure:** If the report is not generated due to technical issues, the system will ask the user to retry the process.

5. **Use-case name:** Manage Forms

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Admin

Secondary actor: System (Centralized Database)

Stakeholders and interest:

i. **Admin:** interested in managing patient forms such as operation, consent, discharge and emergency forms

Pre-condition: Admin has logged and has privileges required to manage the patient forms.

Post-condition: The forms can be created, updated, viewed and stored in the database.

Main success scenario:

Actor	System
1. Admin logs in	
2. Admin navigates to the 'Patient Forms' section	3. System redirects to the patient forms section
4. Admin selects a specific type of form	5. System then displays the form
6. Admin clicks the 'Print form' button	
7. Admin can select to view, delete or update the existing forms.	

Extensions:

i. **Technical Issue:** If due to any technical issue the form is not saved, the system displays a message saying "Technical Issue: Try again"

ii. **Delete Form:** The system will ask for confirmation before permanently deleting the form

6. **Use-case name:** Generate Slips

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Admin

Secondary actor: System (Centralized Database)

Stakeholders and interest:

i. **Admin:** interested in generating slips for in-person appointment booking

ii. **Patient:** interested in receiving a slip for the appointment

Pre-condition: Admin has logged and has privileges required to generate slips.

Post-condition: Slip is generated and can be printed or downloaded.

Main success scenario:

Actor	System
1. Admin logs in	
2. Admin navigates to the 'Generate Slip' section	**3.** System redirects to the generate slip section
4. Admin enters the patient ID	**5.** System fetches the patient information from the database
6. Admin clicks the 'Generate Slip' button	**7.** System generates the slip with necessary details
8. Admin can click the 'Print Slip' button	

Extensions:

i. **No patient found:** If the entered patient ID is not found then the admin will have to register the patient first

ii. **Slip Generation Failure:** If the slip is not generated due to technical issues, the system will ask the user to retry the process.

7. **Use-case name:** View Medical Record

Scope: Sehat e Tafseel

Level: User-goal

Primary actors:

1. **Admin**
2. **Doctor**
3. **Patient**

Stakeholders and interest:

i. **Admin:** interested in viewing the medical records of all the patients for administrative purposes

ii. **Doctor:** interested in viewing the medical records of the patients he/she is consulting

iii. **Patient:** interested in viewing their own medical records

Pre-condition: The user has logged in the system with privileges and has the ID to access the patient record

Post-condition: Complete medical record of the searched ID is displayed

Main success scenario:

Actor	System
1. The user has logged in	
2. The user navigates to the 'View Medical Record' section	3. System redirects to the medical record page
4. The user enters the patient ID	5. System checks if the user has entered a valid ID and retrieves the records from the centralized database
	6. System displays the medical record details

	7. System ensures that the information displayed is according to the user's role
8. User views the records	

Extensions:

i. Record not found: If the searched ID is not found, the system will display a message saying "Patient with this ID does not exist"

ii. Data retrieval failure: if the medical record cannot be retrieved due to technical issue, the system will display a message saying "Retry or Contact Technical Support Team"

iii. Privacy Violation: If the patient tries to access the medical record of any other patient, the system will display a message saying "Access denied"

8. **Use-case name:** Generate Prescriptions

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Doctor

Stakeholders and interest:

i. **Doctor:** interested in issuing prescriptions for their patients

ii. **Patient:** interested in receiving a prescription for their treatment

Pre-condition: Doctor has logged in and has access to the patient's medical record.

Post-condition: prescription is issued and stored in the centralized database

Main success scenario:

Actor	System
1. Doctor logs in	
2. Doctor navigates to the 'Generate Prescription' mention	3. System redirects to the generate prescription page
4. Doctors enters the patient ID for whom the prescription is to be added	5. System checks if valid ID has been entered
	6. System retrieves complete medical history of the patient
7. Doctor enters the prescription details and submits the prescription	8. System saves the prescription in the database.
	9. System prompts a message asking the doctor to print the prescription

Extensions:

i. **Invalid prescription details:** if the doctor enters invalid or incomplete details, the system will display a message saying "The prescription needs correction"

ii. **Patient record not found:** if the entered ID is incorrect, the system will display an error message saying "Patient with this ID does not exist"

iii. **Prescription generation failure:** If there is a technical issue while generating the prescription, the system will display a message saying "Retry the prescription generation process"

9. **Use-case name:** Schedule Consultations

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Doctor

Stakeholders and interest:

i. **Doctor:** interested in scheduling in-person consultations with their patients

ii. **Patient:** interested in booking consultations with the doctor

Pre-condition: Doctor is logged into the system and has access to the defining the consultation slots

Post-condition: The consultation is scheduled and a notification is sent to the patient.

Main success scenario:

Actor	System
1. Doctor logs in	
2. Doctor navigates to the 'Schedule Consultations' section	**3.** System redirects to the schedule consultation page
4. Doctor adds the free slots	
5. Doctor confirms the booking of consultations made by the patients	**6.** System validates the selected slot and schedules the consultation.
	7. System notifies the patient of booking confirmation

Extensions:

i. Technical issue: If there is a technical issue while scheduling/ confirmation, the system displays a message saying "Retry confirming the booking"

ii. Double booking: If the doctor accidentally confirms overlapping consultations, the system detects it and prevents double booking

10. **Use-case name:** Add Remarks

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Doctor

Stakeholders and interest:

i. **Doctor:** interested in adding remarks or notes on patient's previous treatments

ii. **Patient:** interested in having their medical history updated for follow-up consultations

Pre-condition: Doctor has logged in and has access to the patient's medical records and full history

Post-condition: Remarks are successfully added to the patient's treatment history and accurately saved in the database

Main success scenario:

Actor	System
1. Doctor logs in	
2. Doctor navigates to the "View Medical Record" section	3. System redirects to the view medical record page
4. Doctor searches for the patient with their unique ID	5. System displays complete medical history
6. Doctors selects the relevant entry to add a remark	7. System will open an interface to add remarks
8. Doctor enters remarks and submits the remark	9. System saves the remark in the database and sends confirmation

	message saying “Remark successfully added”

Extensions:

i. **Record not found:** If the searched ID is not found, the system will display a message saying “Patient with this ID does not exist”

ii. **Data retrieval failure:** if the medical record cannot be retrieved due to technical issue, the system will display a message saying “Retry or Contact Technical Support Team”

11. **Use-case name:** Identify Disease

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Patient

Secondary actor: Healthbot (System)

Stakeholders and interest:

i. **Patient:** interested in receiving diagnosis based on their symptoms and getting doctor recommendations

ii. **Doctor:** interested in treating the patients based on healthbot recommendations

Pre-condition: The patient has logged in and has access to the Healthbot.

Post-condition: The Healthbot identifies the possible disease(s) based on the symptoms provided and then recommends the relevant doctors(s).

Main success scenario:

Actor	System
1. Patient logs in	

2. Patient navigates to the 'Healthbot' section	3. System redirects to the healthbot page
4. Patient enters their symptoms	5. System analyzes the symptoms and identifies the potential disease(s)
	6. System recommends doctors specializing in the identified disease
7. Patient reviews the identified disease and the recommended doctor(s)	
8. The patient can book appointment with the recommended doctor	

Extensions:

i. **Insufficient symptoms:** If the patient provided too few symptoms, then the system prompts the patient to enter more symptoms

ii. **No disease identified:** If the healthbot is not able to match the symptoms with any disease then prompt the patient "unable to identify the disease'

iii. **Technical issue:** if the healthbot fails to process the symptoms, the system display a message "Technical Issue: Try again"

12. Use-case name: Book Appointment

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Patient

Stakeholders and interest:

i. **Patient:** interested in booking an appointment with a doctor

ii. **Doctor:** interested in receiving appointment bookings

Pre-condition: The patient has logged in and has access to the doctor's schedule and availability.

Post-condition: The appointment is successfully booked and saved in the centralized database,

Main success scenario:

Actor	System
1. Patient logs in	
2. Patient opens the "Book Appointment" section	3. System redirects to the book appointment page
4. Patient searched for a doctor by name of by specialization	5. System displays the list of available doctors based on the filter check
6. Patient selects a preferred time from the slots available	7. System saves the appointment and notifies the doctor

Extensions:

i. **No doctor available:** if the searched doctor or specialization is not available, the system displays a message saying "Doctors not available"

ii. **Unavailable time slots:** if the patient searches for a doctor or the doctor is recommended by the healthbot and there are no free slots available, the system will display a message saying "No free slots available at the moment"

iii. **Appointment Failure:** if the patient books an appointment but there was a technical issue then the system displays a message saying "Try the booking process again"

13. Use-case name: Receive Health Alerts

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Patient

Stakeholders and interest:

i. **Patient:** interested in receiving timely health related alerts

Pre-condition: The patient is a registered user and their medical record is up-to-date.

Post-condition: The patient receives health alerts regarding their health.

Main success scenario:

Actor	System
1. Patient logs in	2. System generated health alert
3. Patient receives the health alert	
4. Patient acknowledges by opening the notification	5. System confirms the acknowledgement and gives options to take actions based on the alert
6. Patient takes the necessary action	

Extensions:

i. **No health alert available:** If there is no pending health alerts then the system displays a message saying “No health alerts at the moment”

ii. **Missed alert:** If the patient does not acknowledge the alert within 24 hours, the system will send a follow-up alert

14. Use-case name: Skin Disease Detection

Scope: Sehat e Tafseel

Level: User-goal

Primary actor: Patient

Secondary actor: System (Skin Disease Detection System)

Stakeholders and interest:

i. **Patient**: interested in receiving diagnosis of their skin condition by uploading images and receiving recommendation for a doctor

Pre-condition: The patient has logged in and has access to the skin disease detection feature. There should be a trained model for identifying skin diseases.

Post-condition: The model identifies the skin disease and recommends doctor(s) for in-person consultation.

Main success scenario:

Actor	System
1. Patient logs in	
2. Patient navigates to the 'Skin Disease Detection' section	3. System redirects to the skin disease detection page
4. Patient uploads an image of the affected skin	5. System processes the image and identifies the disease
	6. System recommends relevant doctors
7. Patient reviews the diagnosis and doctors recommended	

Extensions:

i. **Poor image quality:** If the patient uploads a poor quality image, then the image displays a message saying “Picture unclear. Upload another picture”

ii. **No disease identified**: If the skin disease detection system is unable to identify the disease then a message is displayed saying “Unable to identify the skin disease”

iii. **Invalid image format:** if the patient uploads an image in a format that is not supported by the system then a message is displayed saying “Image is not in the correct format”

iv. **Technical issue:** if there is a technical issue while uploading the image, the system will display a message saying “Try uploading the image again”

2.2 Functional Requirements

2.2.1 Admin Module

Following are the requirements for module 1:

FR1: The system **shall** allow admins to assign roles (Doctor, Patient, etc.) to users.

FR2: The system **must** provide an interface to modify or revoke user roles and permissions.

FR3: The system **shall** allow the admin to update and delete patient records.

FR4: The system **must** ensure the security of sensitive patient information.

FR5: The system **shall** generate slips for patients with a unique reference ID based on their CNIC.

FR6: The system **must** store and track unique IDs for each patient.

FR7: The system **shall** allow admins to manage different types of forms (application, operation, discharge, and consent forms).

FR8: The system **must** ensure that all forms are linked to patient records for easy retrieval.

FR9: The system **should** allow the admin to view and analyze medical data for predictive insights.

FR10: The system **shall** provide graphical reports for trends and analysis of patient data.

2.2.2 Patient Module

Following are the requirements for module 2:

FR1: The system **shall** allow patients to view their medical history, including diagnoses, treatments, medications, prescriptions, lab reports and surgeries.

FR2: The system **must** restrict access to only the authenticated patient's medical records.

FR3: The system **shall** allow patients to interact with the Healthbot for symptom entry and health assistance.

FR4: The system **should** provide patients with possible disease diagnoses based on their symptoms.

FR5: The system **must** allow patients to search for doctors by specialty.

FR6: The system **shall** display the availability and profiles of doctors matching the search criteria.

FR7: The system **should** allow patients to view relevant health blogs and articles.

FR8: The system **shall** allow patients to book an appointment with doctors.

FR9: The system **must** confirm the booking and send an appointment slip to the patient.

FR10: The system **shall** allow patients to upload skin images for analysis.

FR11: The system **should** return possible skin disease/allergy diagnoses and recommend doctors.

2.2.3 Doctor Module

Following are the requirements for module 3:

FR1: The system **shall** allow doctors to view the medical records of their assigned patients.

FR2: The system **must** restrict doctors' access to only their assigned patients.

FR3: The system **shall** allow doctors to update patient records after consultation or treatment.

FR4: The system **must** keep a log of all changes made by doctors for auditing purposes.

FR5: The system **shall** allow doctors to add remarks or additional notes to patient records.

FR6: The system **shall** allow doctors to generate prescriptions for patients.

FR7: The system **must** store prescriptions in the patient's record for future reference.

2.2.4 Centralized Database Module

Following are the requirements for module 4:

FR1: The system **shall** store comprehensive patient data, including diagnoses, treatment, prescriptions, surgeries, and other lab reports.

FR2: The system **must** ensure the integrity and consistency of patient records.

FR3: The system **shall** allow hospitals to access patient records as needed, based on user permissions.

FR4: The system **must** maintain a real-time database for immediate access to updated information.

FR5: The system **must** enforce strict access control mechanisms to protect the confidentiality of patient data.

FR6: The system **shall** utilize encryption to secure data during transmission and storage.

FR7: The system **shall** allow real-time access to patient data for updates and retrievals by authorized users.

2.2.5 Healthbot Module

Following are the requirements for module 5:

FR1: The system **shall** allow patients to input their symptoms into the Healthbot for analysis.

FR2: The system **should** analyze the patient's symptoms and **shall** suggest possible diseases based on pre-trained models.

FR3: The system **shall** recommend doctors based on the identified disease, providing patients with relevant options for consultation.

2.2.6 Skin Disease Detection Module

Following are the requirements for module 6:

FR1: The system **shall** allow patients to upload images of skin conditions for analysis.

FR2: The system **must** ensure that the uploaded images are stored securely.

FR3: The system **should** use computer vision techniques to analyze the image and return possible skin disease/allergy diagnosis.

FR4: The system **shall** recommend doctors based on the results of the skin disease analysis.

2.3 Non-Functional Requirements

2.3.1 Reliability

REL-1: The system **shall** guard against counter measure options that provide automatic logs/alerts which identifies any false detection immediately to the admin.

REL-2: The system **shall** auto-recover from minor failures within 2 minutes to prevent data loss.

REL-3: In the case of critical failure (e.g., database connection failure), the system **must** trigger a backup recovery system within 10 minutes, ensuring that patient medical records and consultation data are not lost.

REL-4: The system **must** utilize regular backups to mitigate the risk of data corruption or failure. A daily backup strategy available to help restore the integrity of the system in the event of any failure.

2.3.2 Usability

USE-1: The system **shall** provide an easy to use user interface so that the users (Admin, Patients, Doctors) can learn about it and navigate things within 30 minutes.

USE-2: The system **must** offer comprehensive help documentation, including tooltips that guide users through core functions like booking appointments, viewing medical records, and assigning roles.

USE-3: The system **shall** validate form inputs in real-time, ensuring that patients cannot submit incorrect or incomplete data (e.g., missing CNIC numbers, invalid date formats).

USE-4: The system **must** comply with accessibility standards (e.g., WCAG 2.1 Level AA) to ensure patients with disabilities can easily interact with the system.

USE-5: The system **shall** allow doctors to retrieve a patient's medical record within 3 clicks, ensuring efficient use of time in critical situations.

2.3.3 Performance

PER-1: The system **shall** process user requests (such as viewing medical records, booking appointments) in under 2 seconds.

PER-2: The system **should** analyze and output the score once an image is uploaded. Images could pertain to anything,(including skin disease detection) in under 20 seconds contrary to current average times of around one minute.

PER-3: System **should** handle up to 10,000 concurrent users without degrading performance, so that patients, doctors and admins can use the system simultaneously in peak hours.

PER-4: Centralized database **shall be** able to support more than 1 million records instantly in less than 1 second.

PER-5: The system **shall** handle real-time updates to patient records, such as doctor remarks and prescription generation, without any noticeable delay (<1 second).

2.3.4 Security

SEC-1: This information **must** be retained, safeguarded and sent in an encrypted (AES-256) manner to prevent access by the patient-sensitive details.

SEC-2: Users (admin, doctor, patient) **shall** be able to authenticate MFA must be imposed so as to minimize unauthorized access.

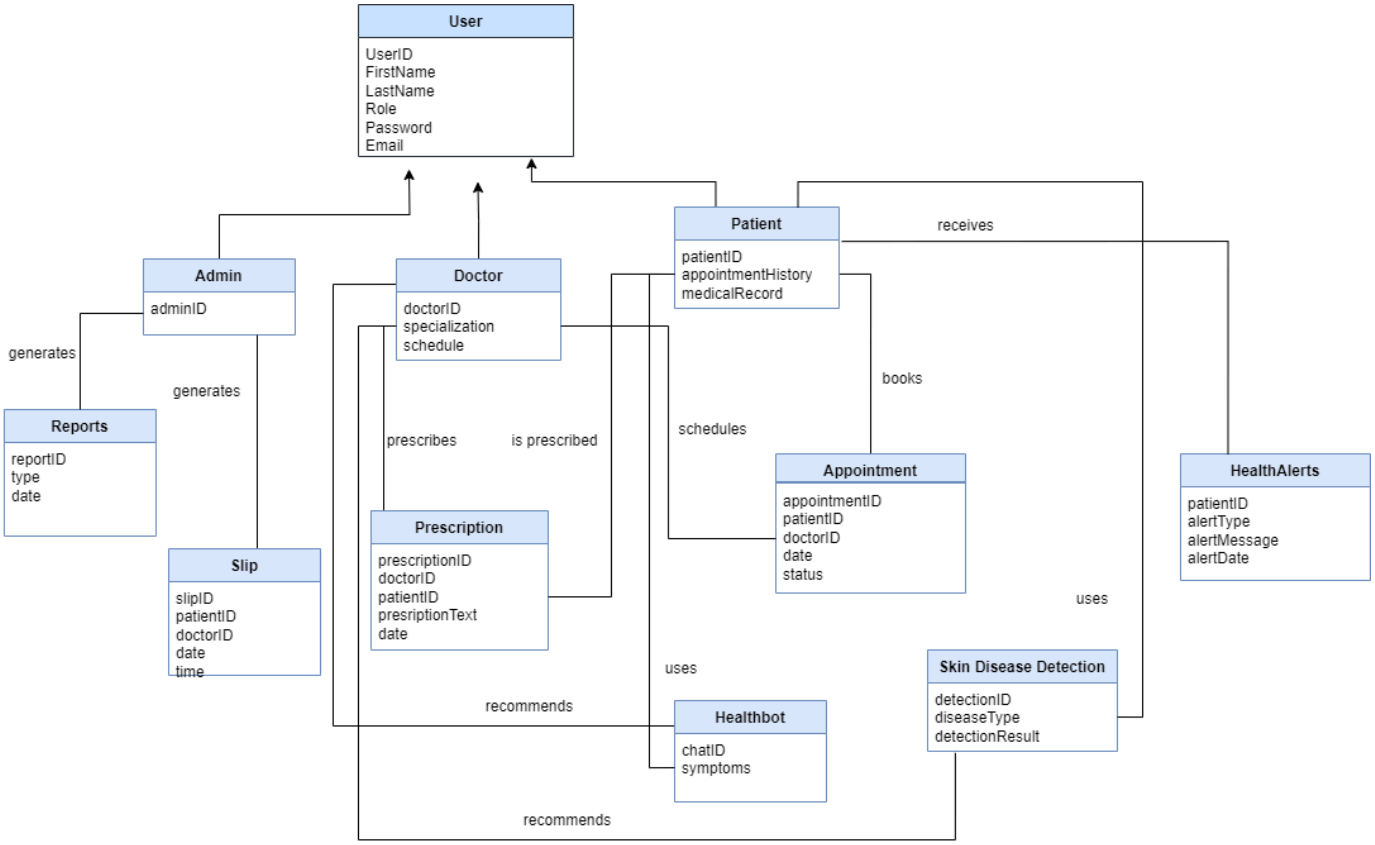
SEC-3: System **must** log all the confidential data (like medical records.) and a system admin should be able to see it using logs.

SEC-4: The system **shall** enforce strict role-based access control (RBAC), ensuring that doctors only have access to their assigned patients' records, and admins control access permissions.

SEC-5: The system **must** have built-in intrusion detection mechanisms to identify and respond to any attempts of unauthorized access within 1 minute of detection.

2.4 Domain Model

Fig 2.2 Domain Model



Chapter 3

System Overview

Sehat-e-Tafseel is designed to serve the needs of the patients and doctors. The Patient interface includes the healthbot, appointment booking and skin disease detection. The healthbot uses NLP to analyze the symptoms and make recommendations. The doctor interface allows the doctors to schedule appointments, add remarks on previous consultations and generate prescriptions. The admin interface is used to manage the roles and permissions and to generate reports.

3.1 Architectural Design

The architectural diagram for our project has been divided into 3 layers: Presentation layer, Application layer and Data Layer.

3.1.1 Presentation Layer

This layer comprises the users, web browser and the mobile phone. The user interacts using the web browser or mobile phone which act as the client interface. Users can input information such as symptoms, view medical records or manage appointments.

3.1.2 Application Layer

This layer is responsible for the core business logic and the communication between the presentation layer and data layer. It includes the following components:

1. Healthbot
2. Doctor recommendation
3. Appointment management system
4. API gateway
5. Image analysis system
6. Notification module

3.1.3 Data Layer

Data storage components are included in this layer. Centralized database will be used along with cloud storage.

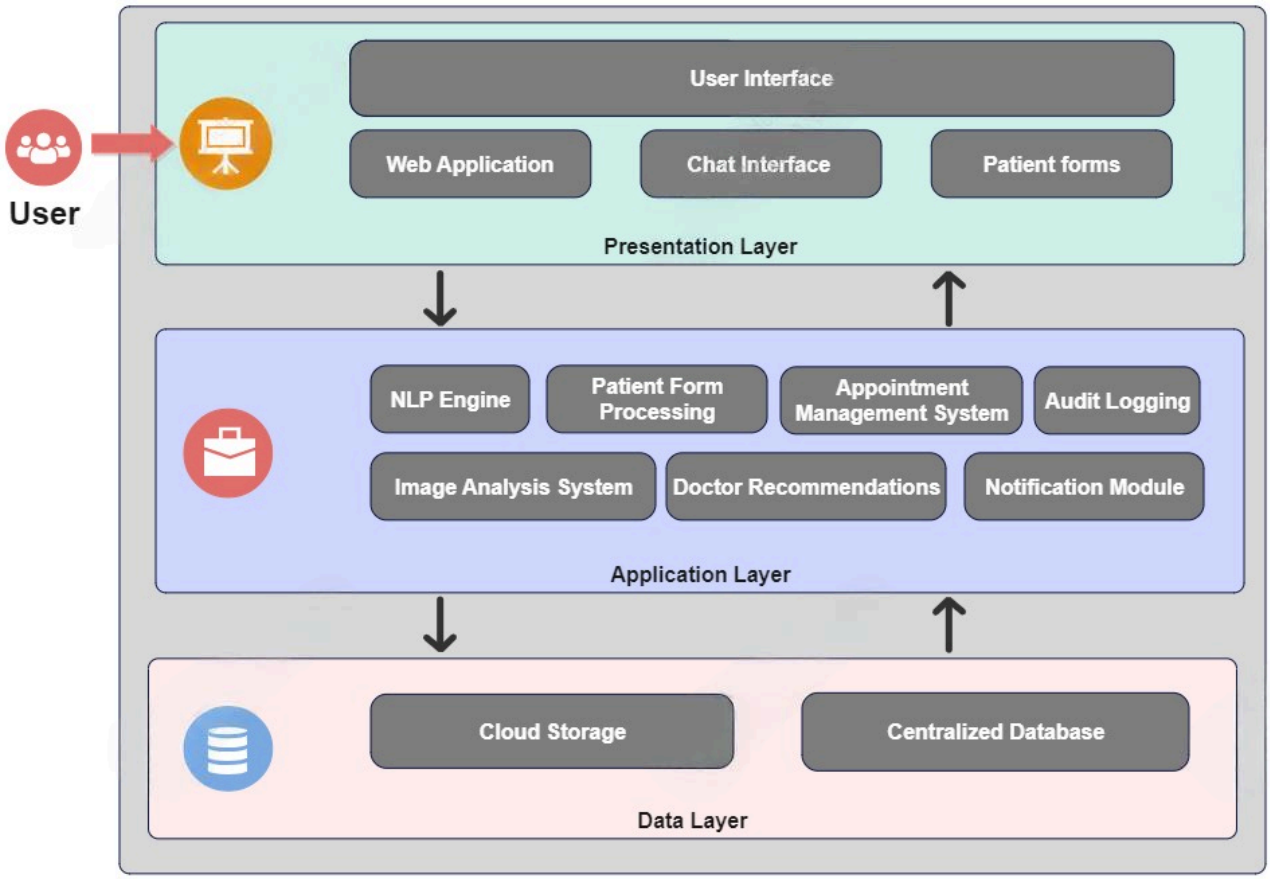


Figure 3.1 Architecture Diagram

3.2 Design Models

Design Models for Object Oriented Development Approach

3.2.1 Activity Diagram

Fig 3.2 Manage Roles and Permissions

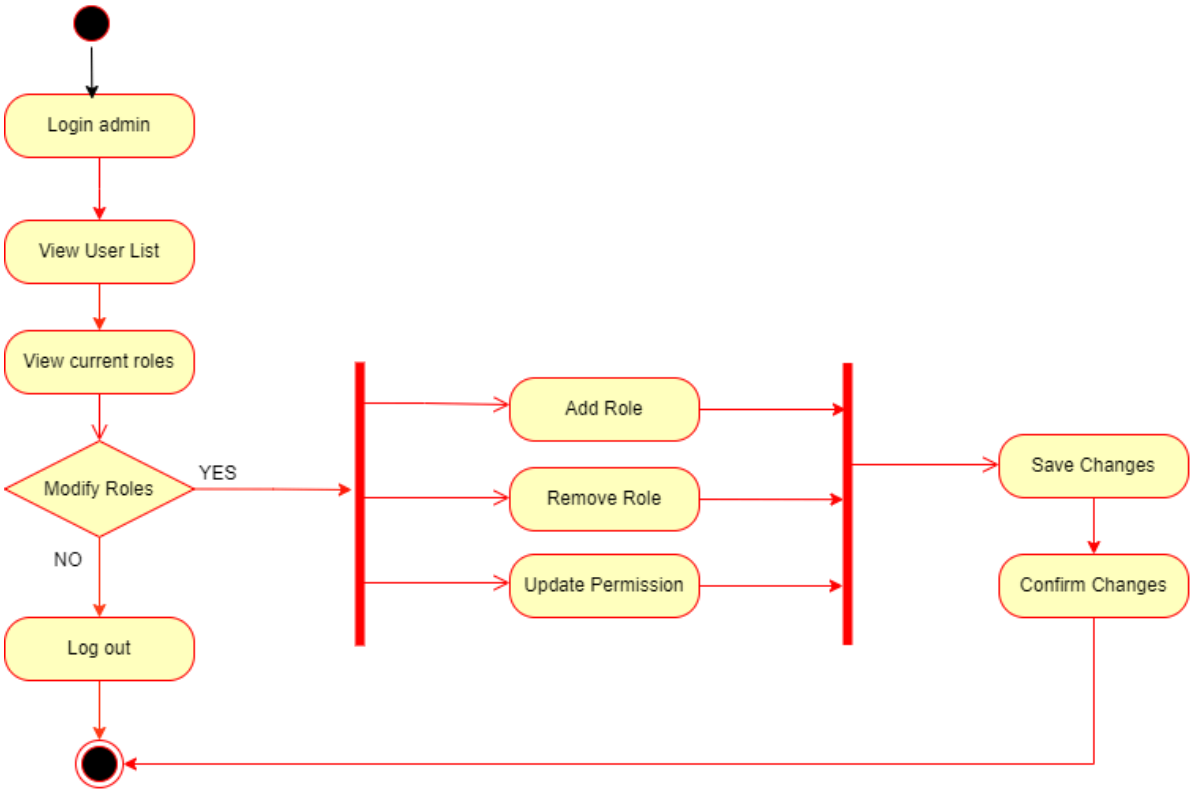


Fig 3.3 Manage Patient Data

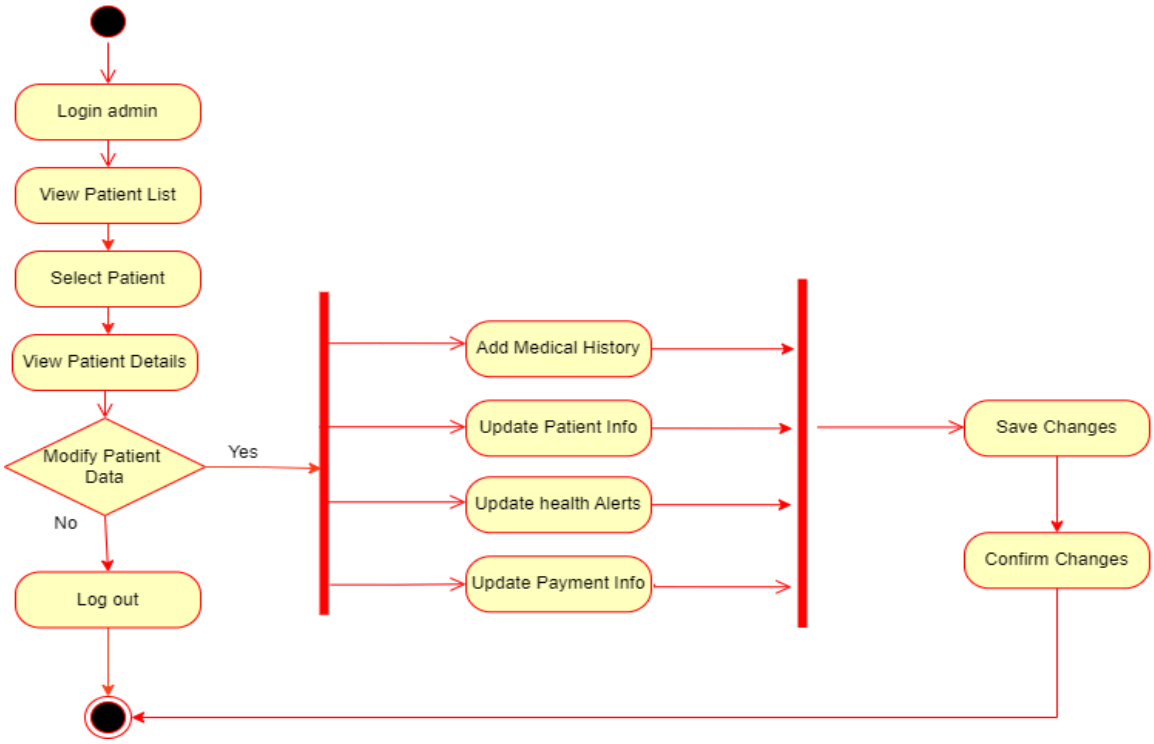


Fig 3.4 Assign Unique IDs

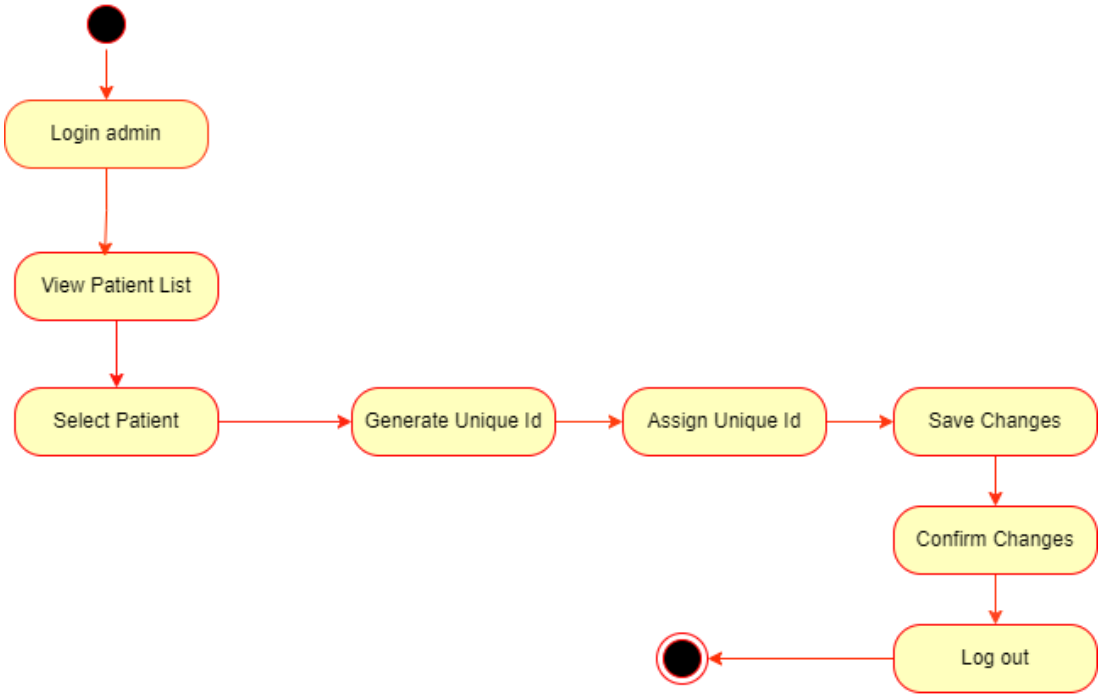


Fig 3.5 Generate Reports

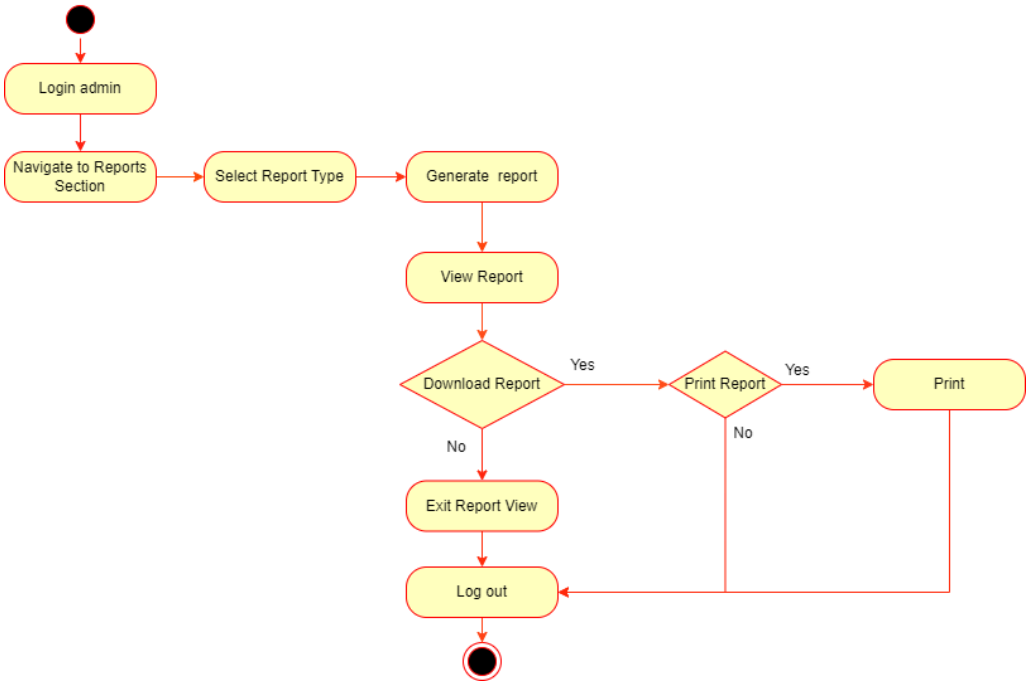


Fig 3.6 Generate Slip

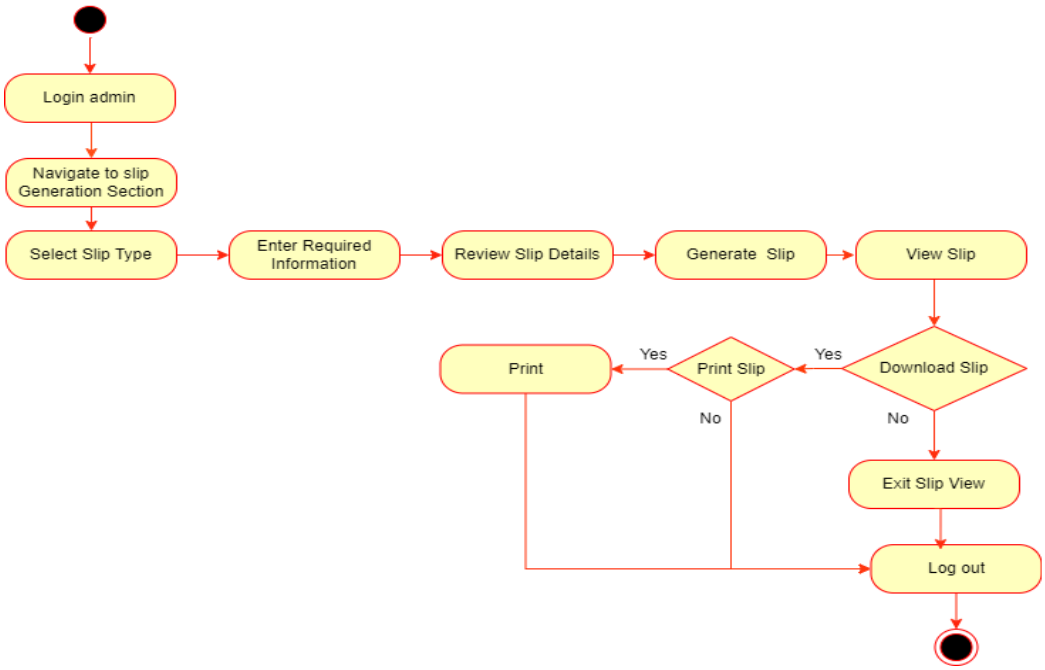


Fig 3.7 View Medical Record

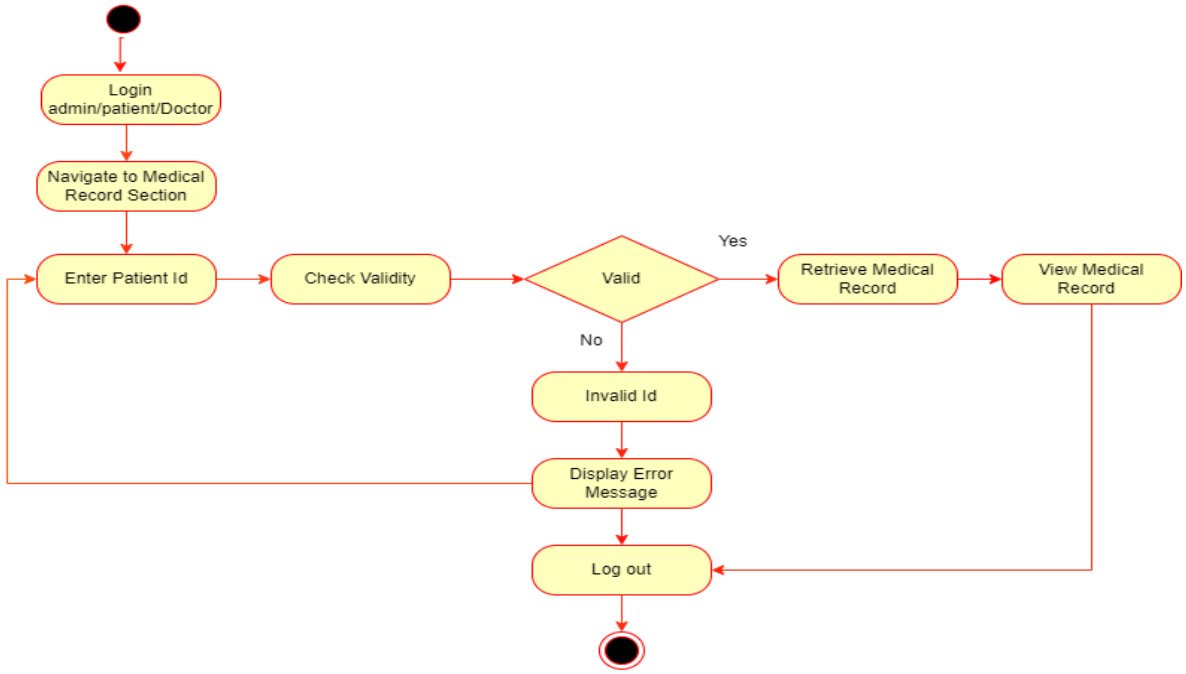


Fig 3.8 Generate Prescriptions

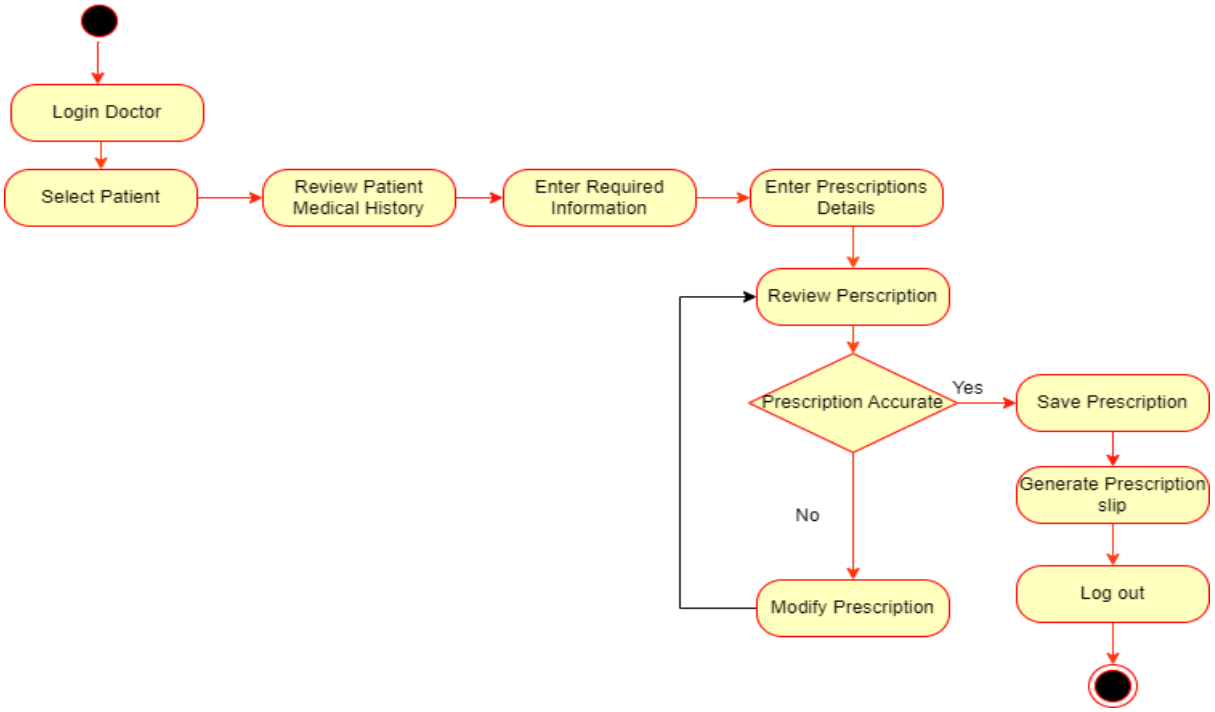


Fig 3.9 Schedule Consultations

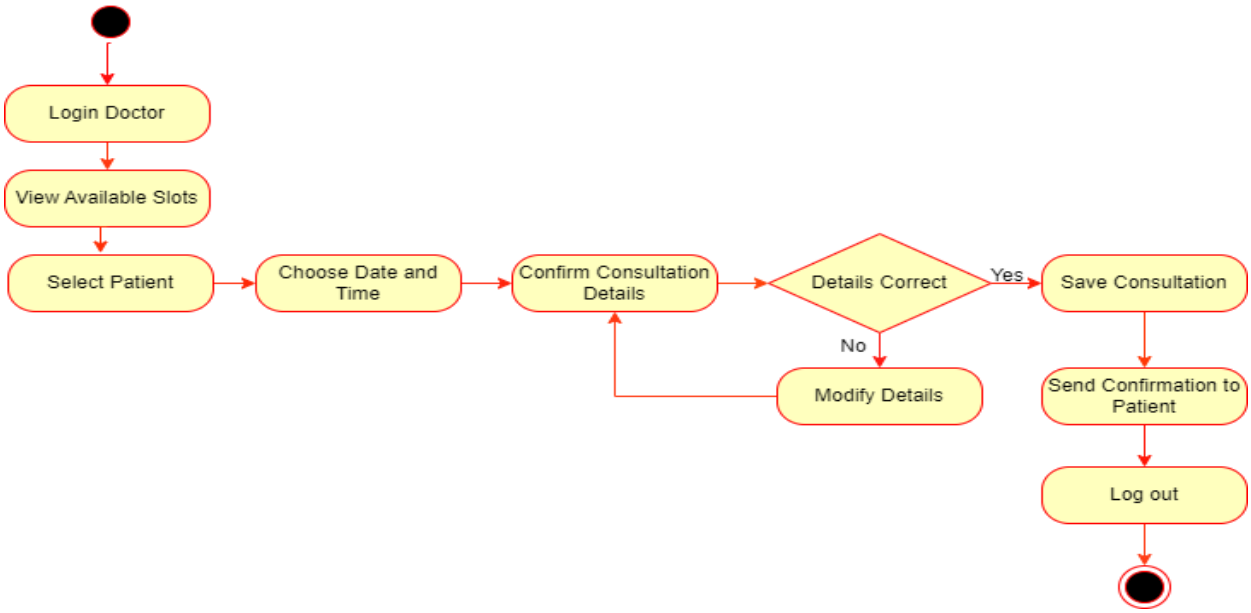


Fig 3.10 Add Remarks

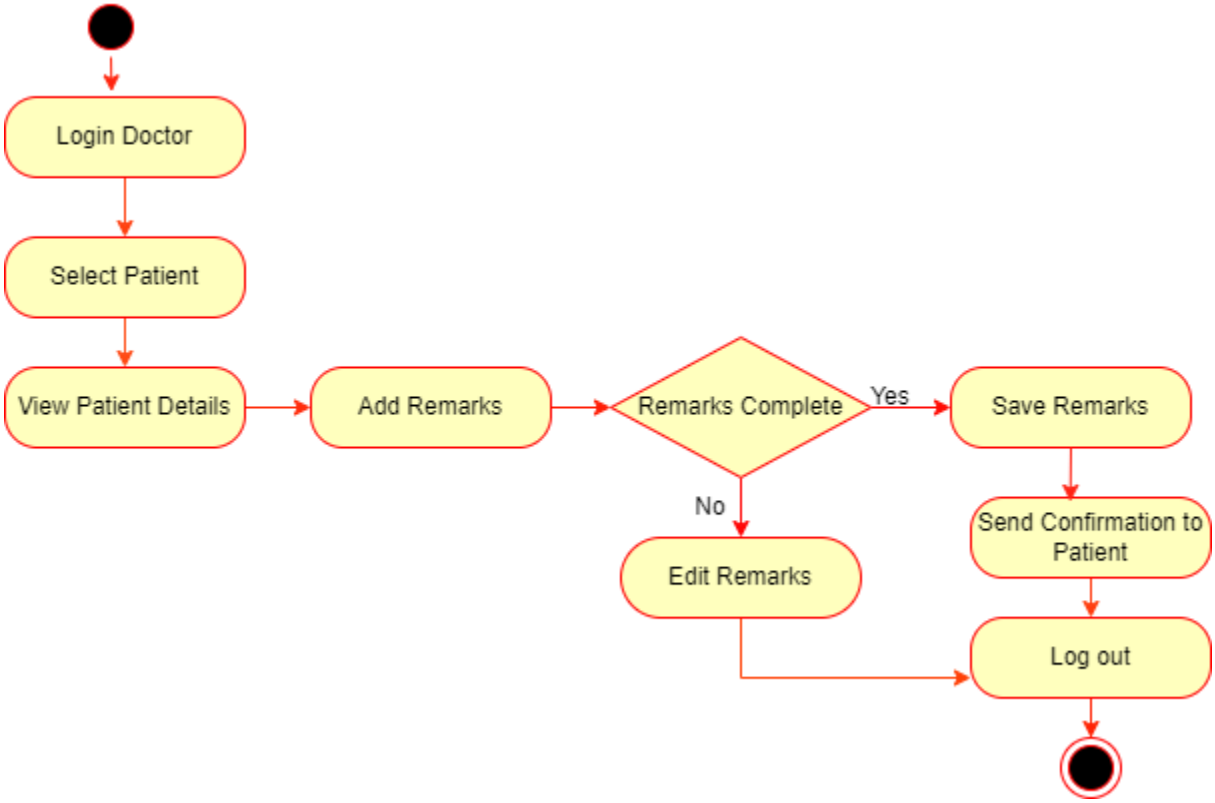


Fig 3.11 Identify Disease

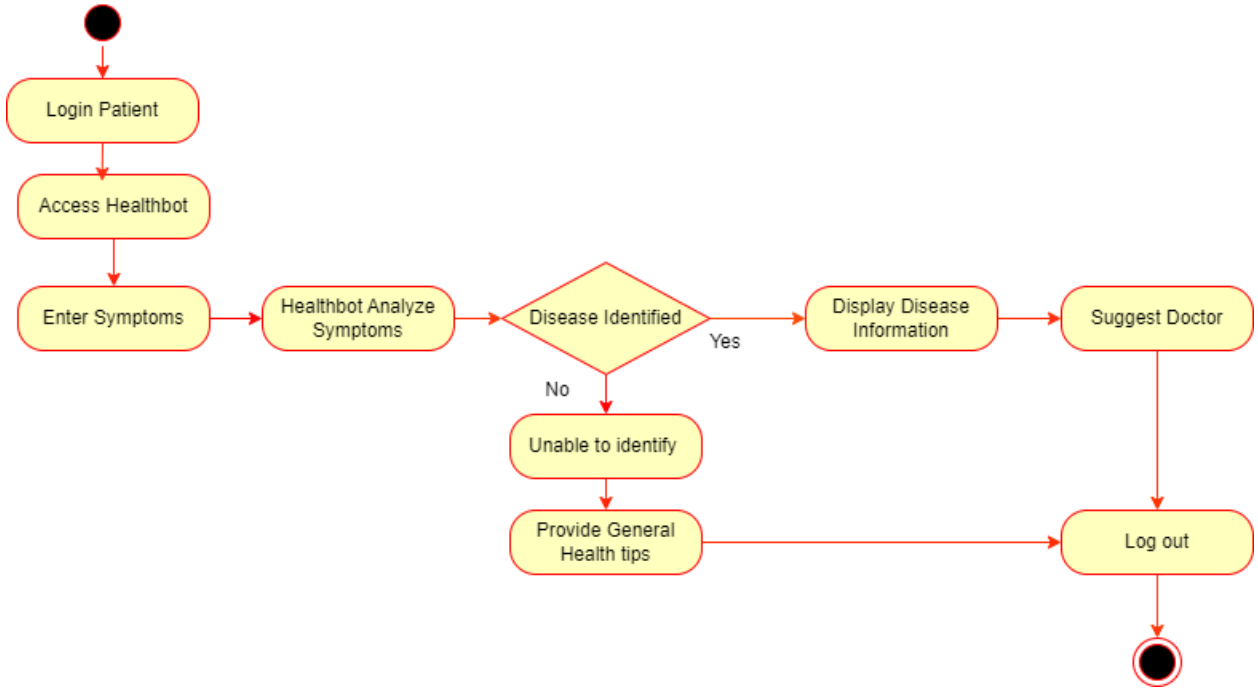


Fig 3.12 Book Appointment

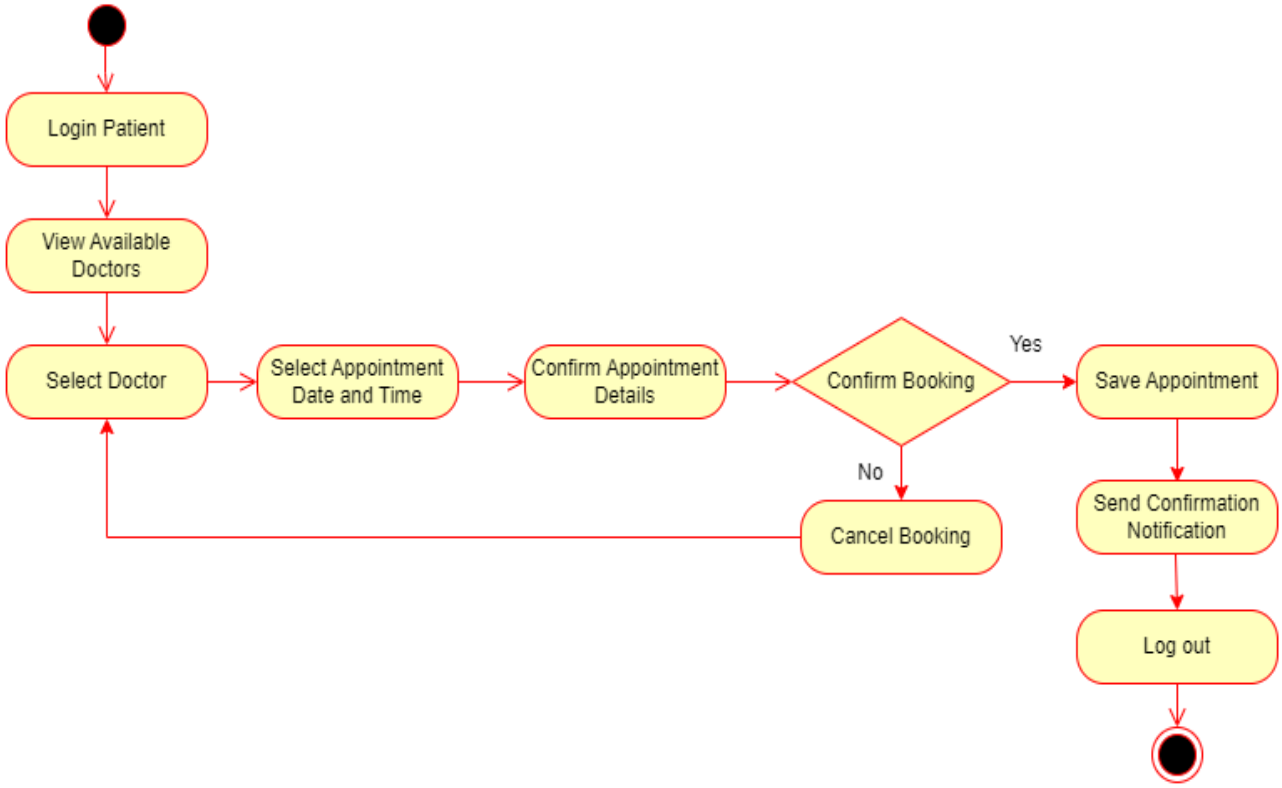


Fig 3.13 Receive Health Alerts

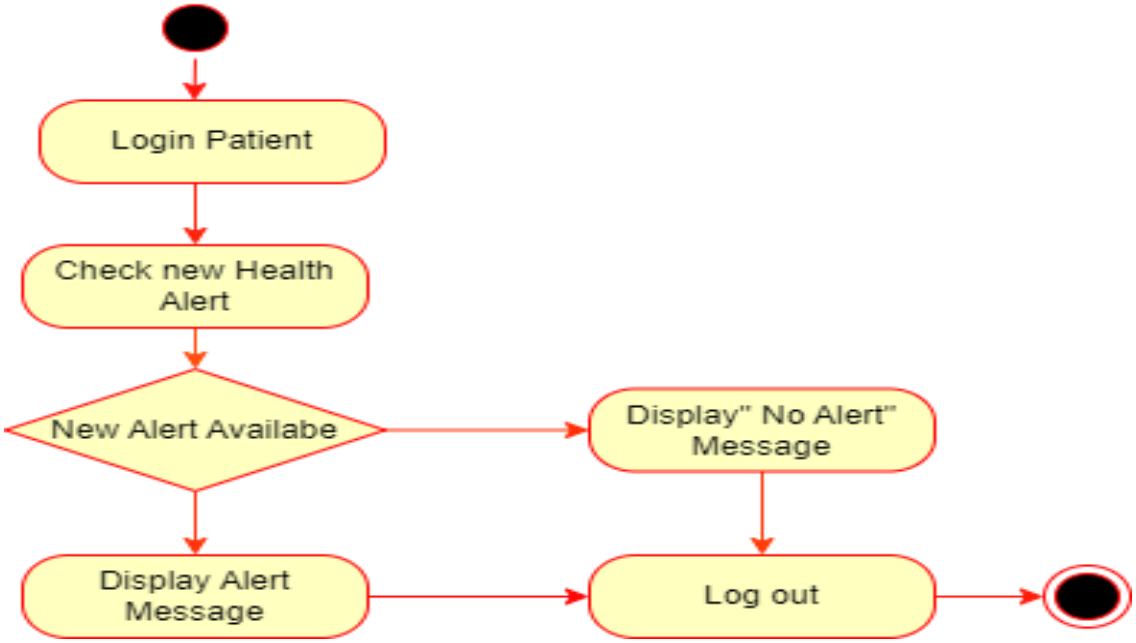
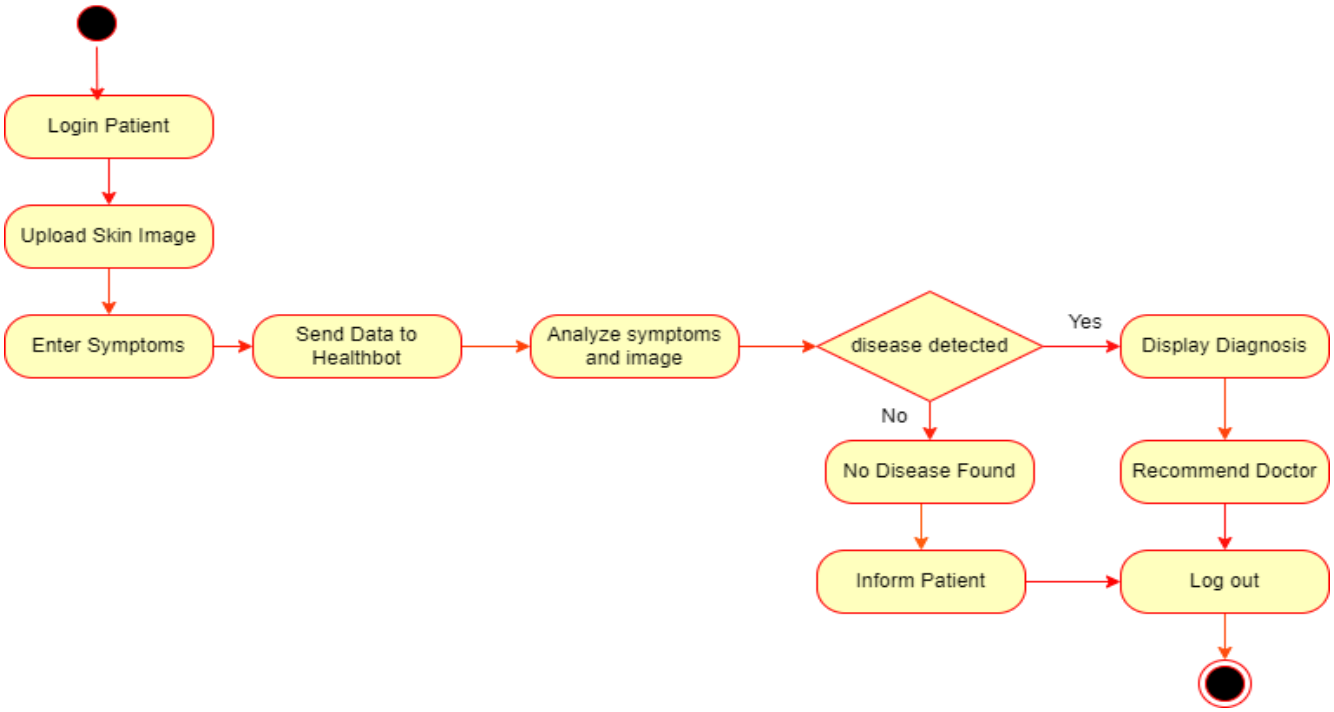
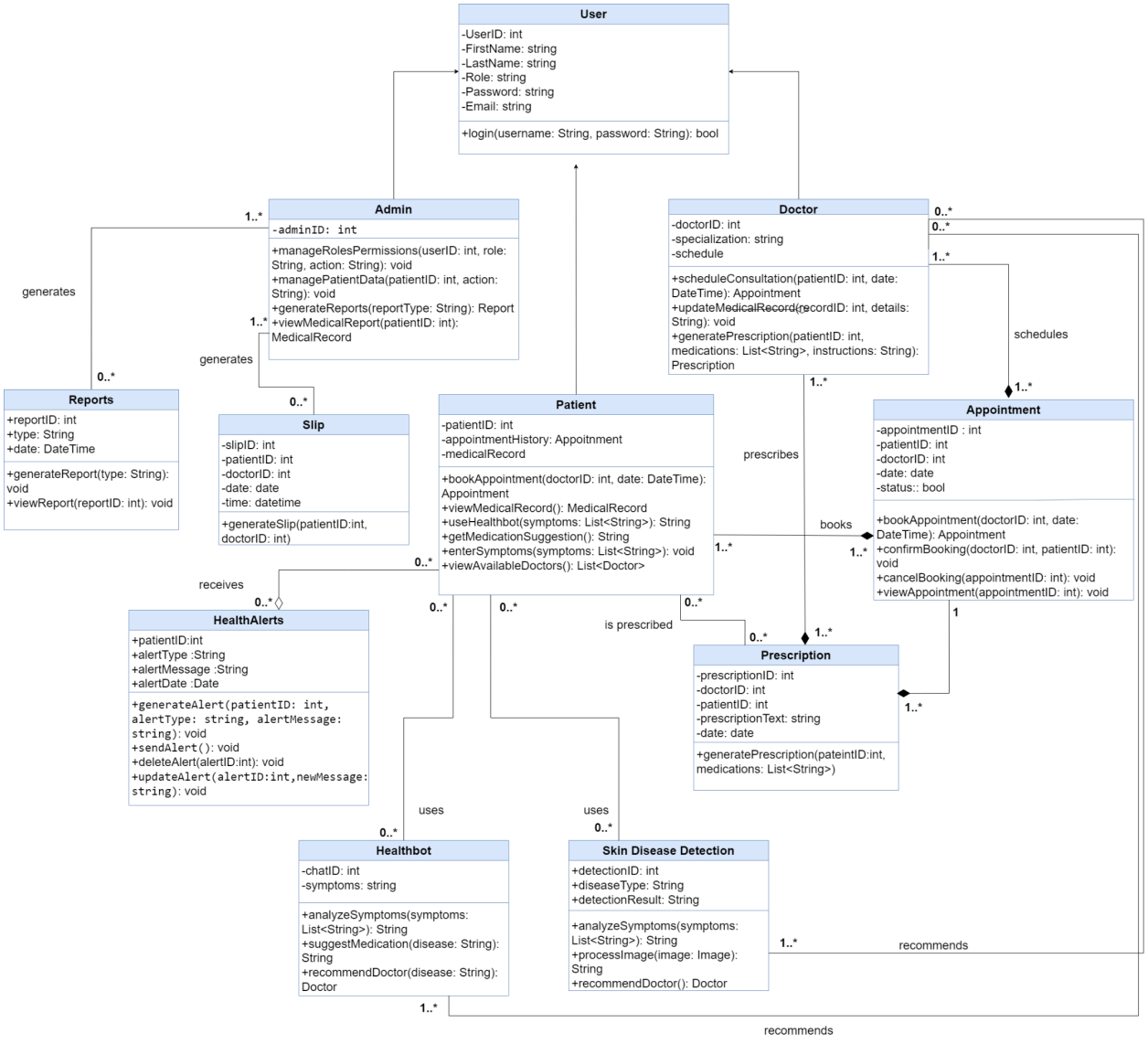


Fig 3.14 Skin Disease Detection



3.2.2 Class Diagram

Fig 3.15 Class Diagram



3.2.3 System Sequence Diagram

Fig 3.16 Manage Roles and Permission

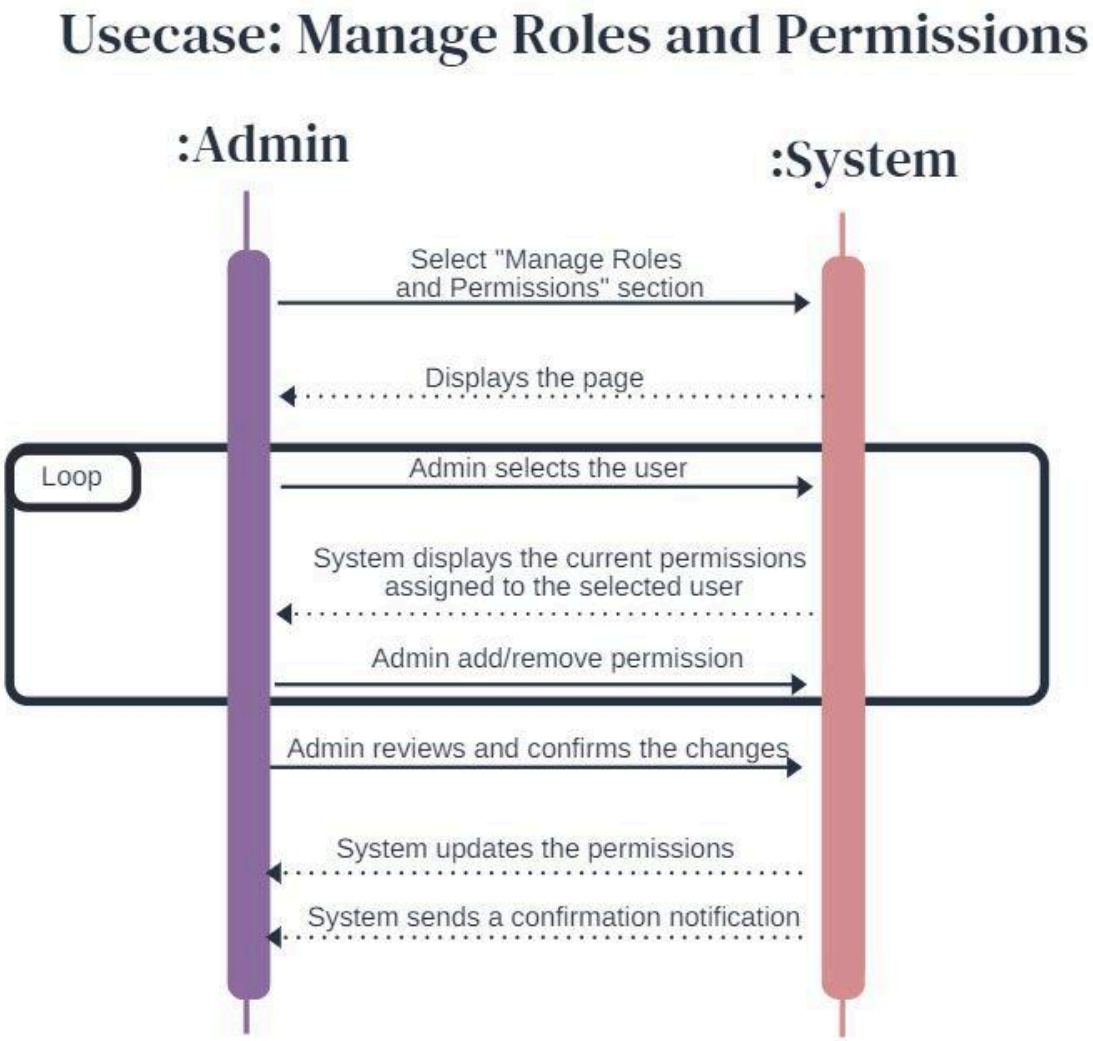


Fig 3.17 Manage Patient Data

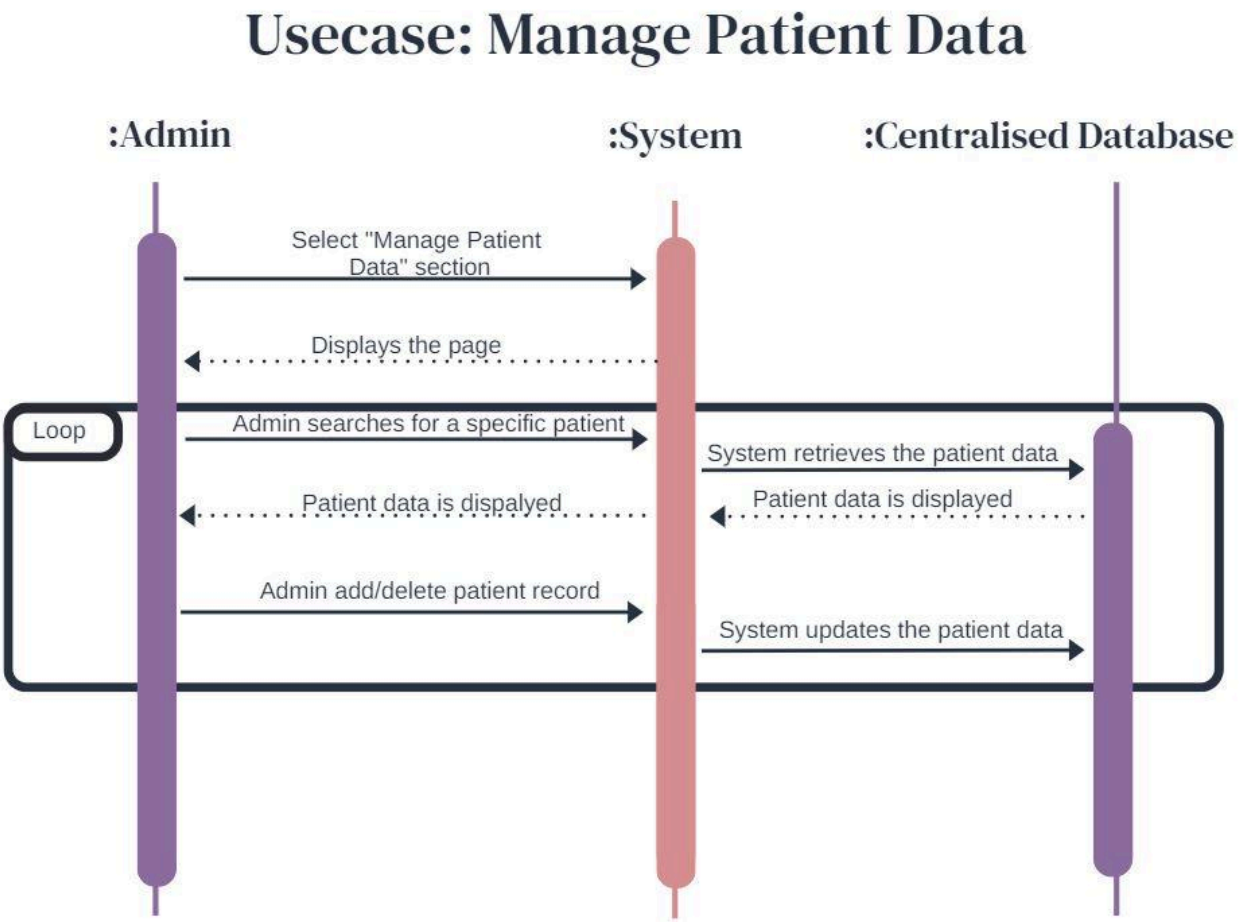


Fig 3.18 Assign Unique IDs

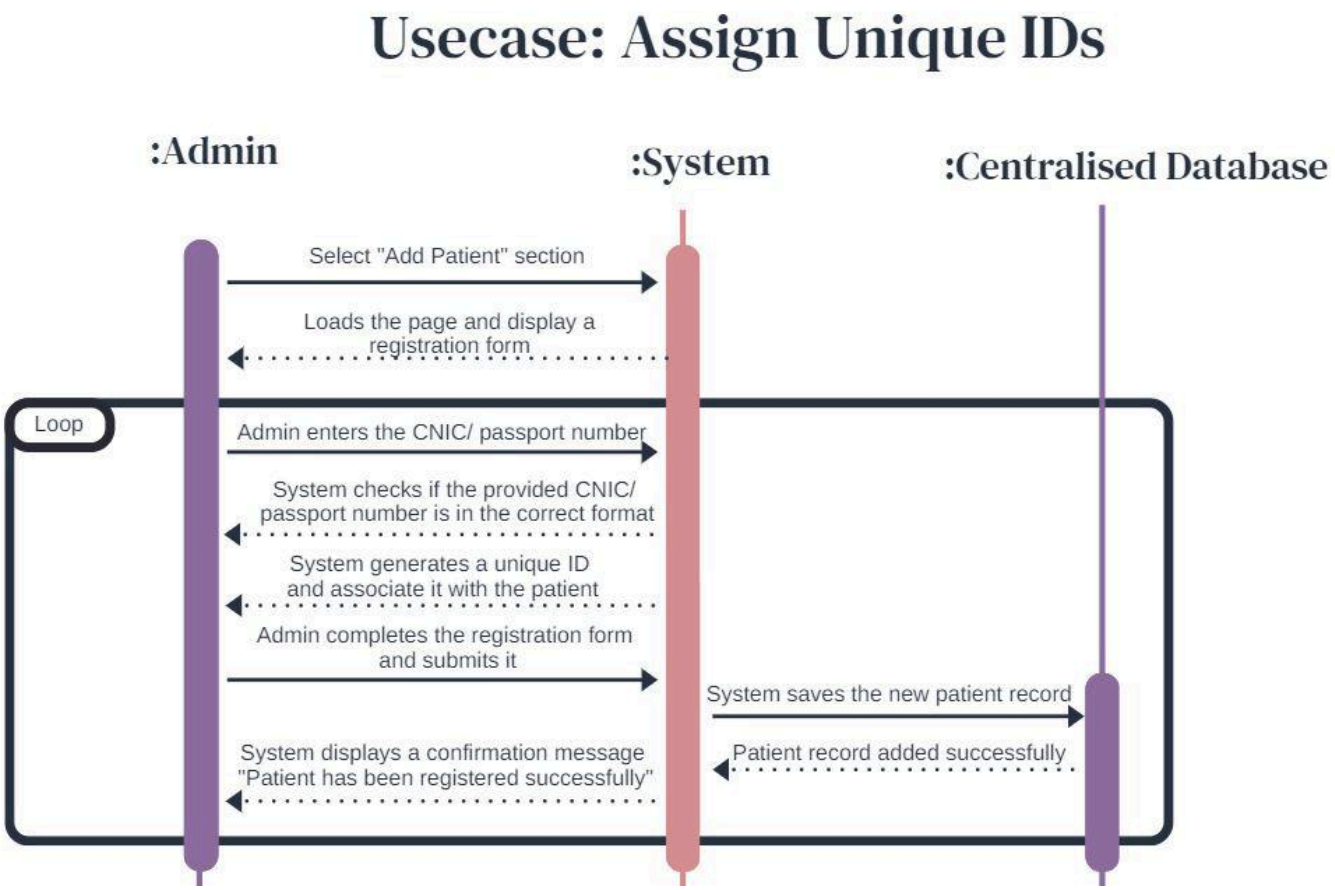


Fig 3.19 Generate Reports

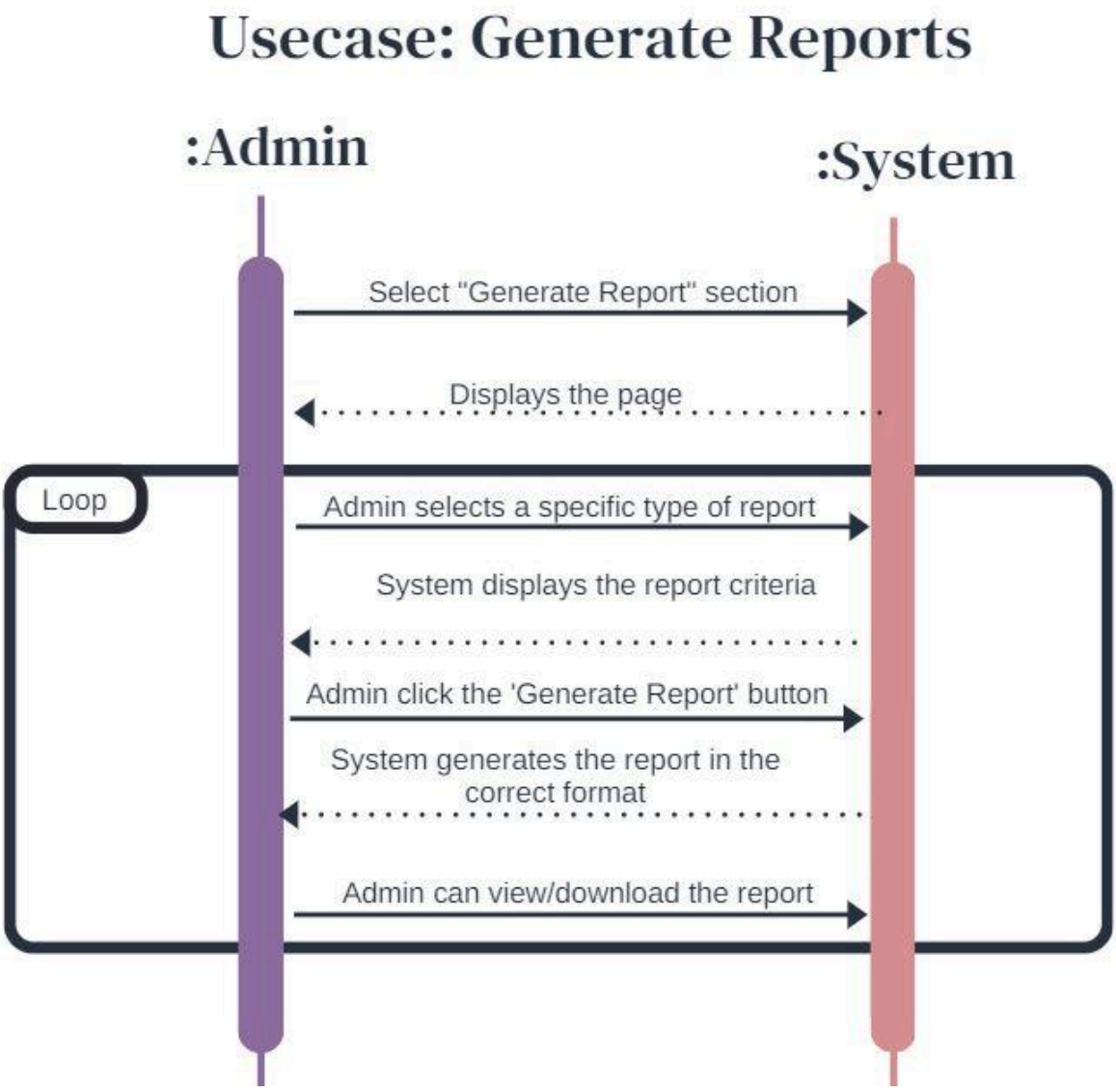


Fig 3.20 View Medical Report

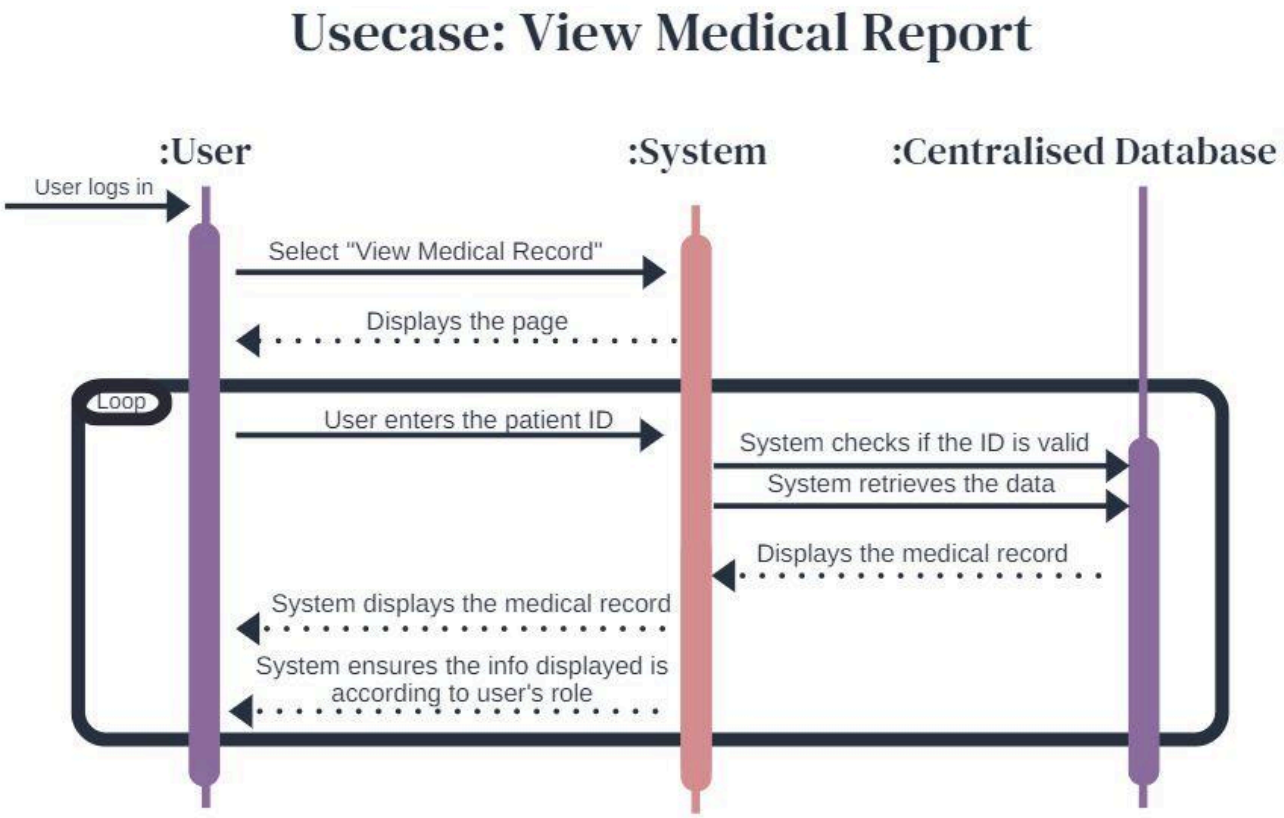


Fig 3.21 Generate Prescription

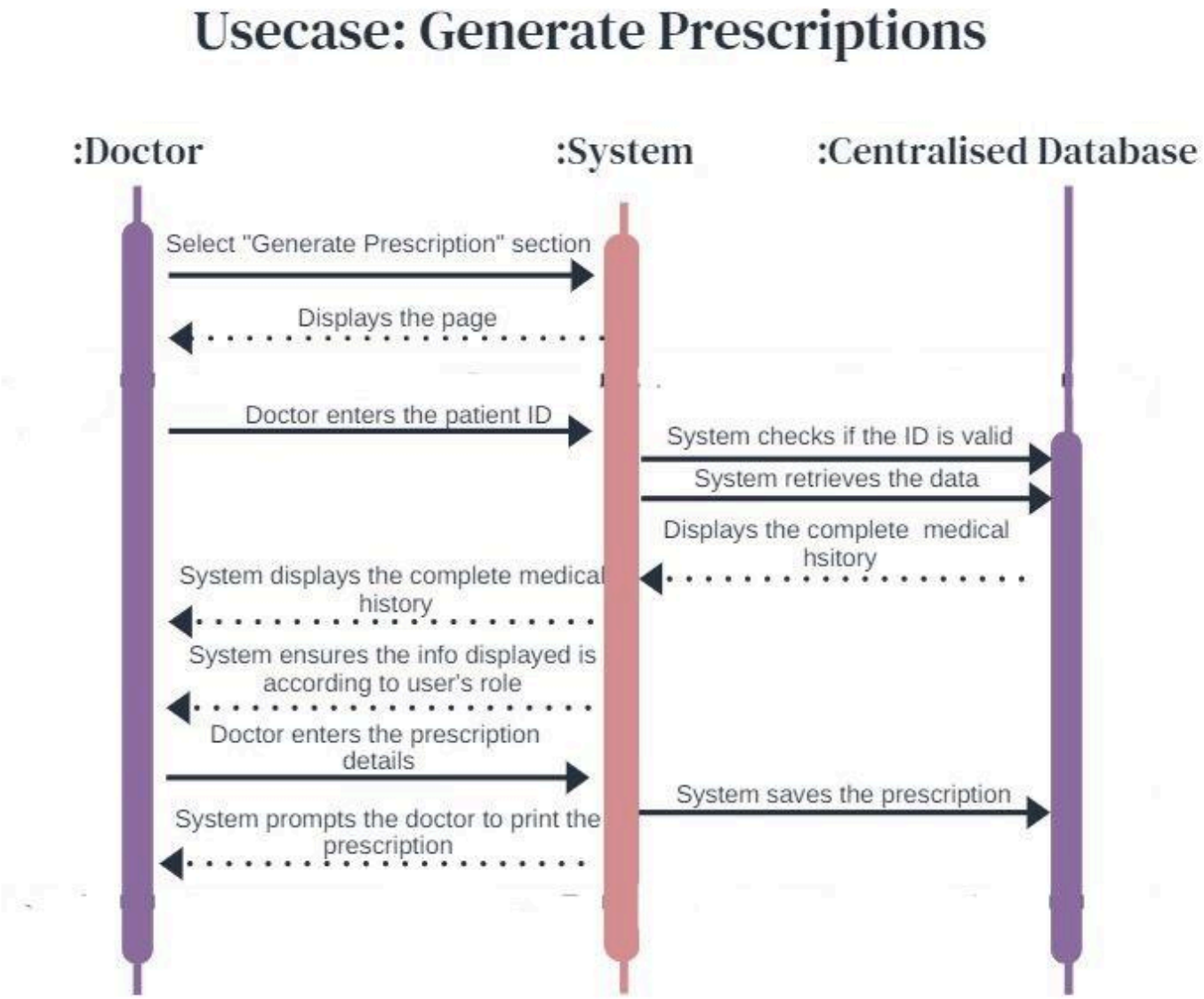


Fig 3.22 Schedule Consultations

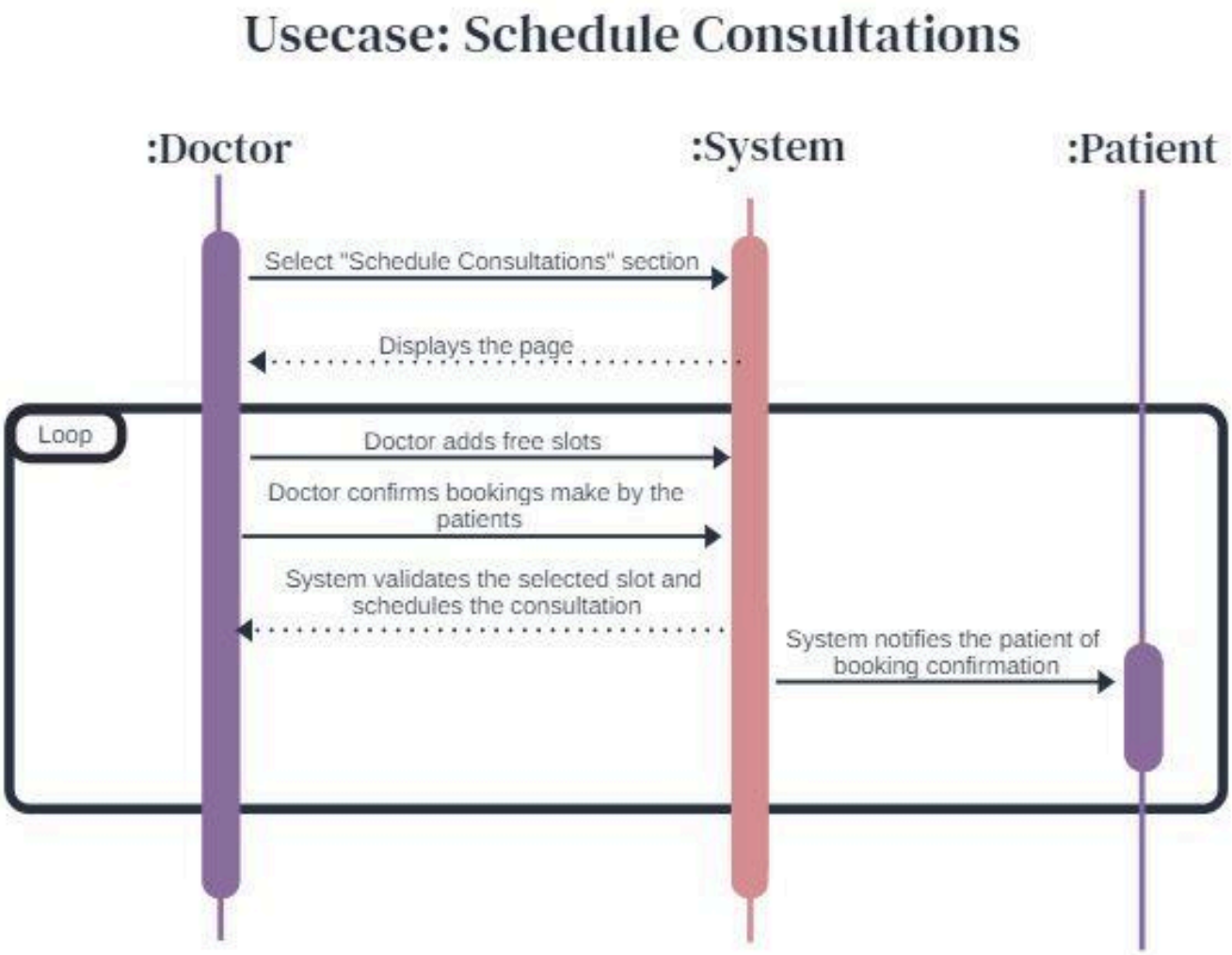


Fig 3.23 Add Remarks

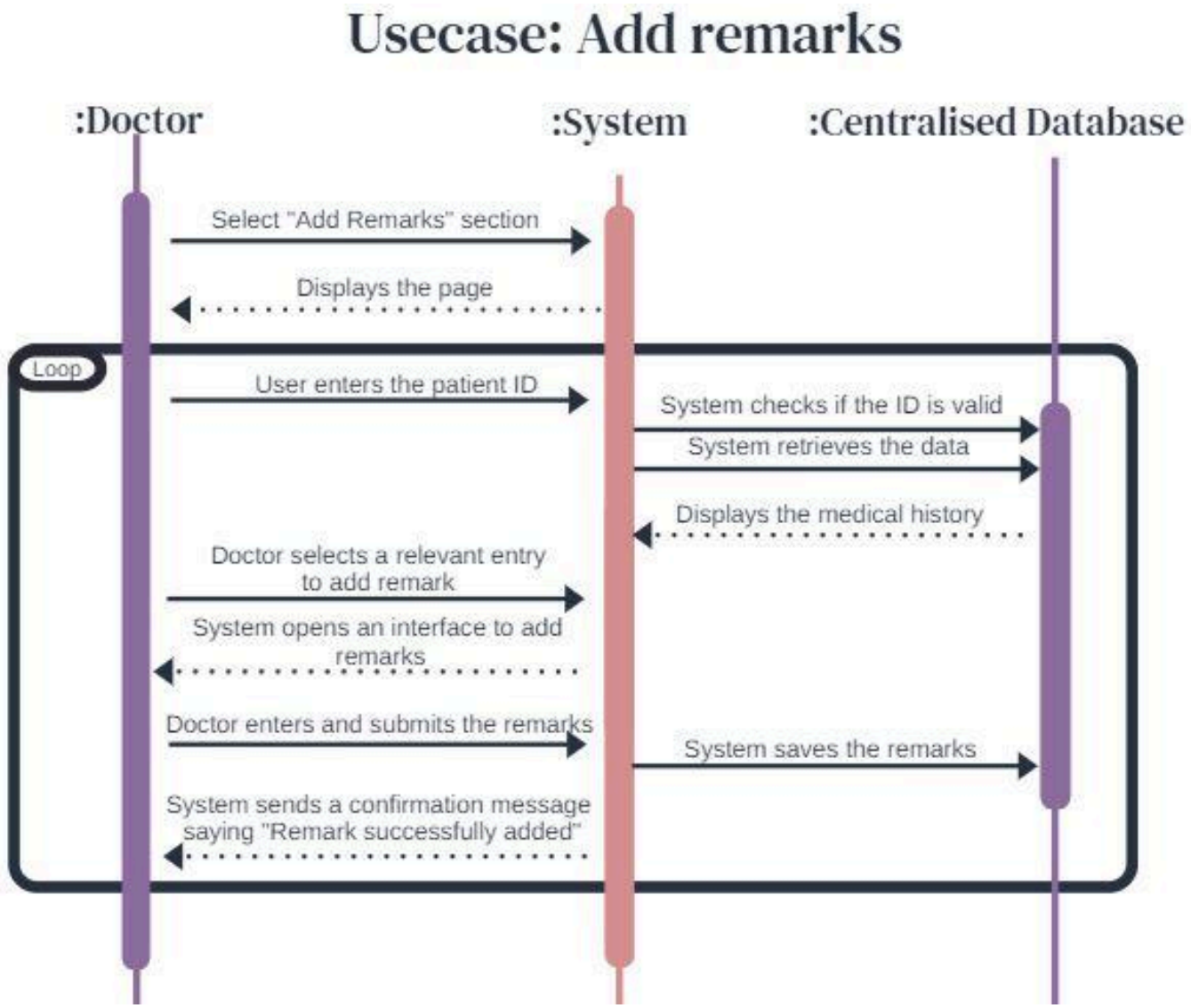


Fig 3.24 Identify Disease

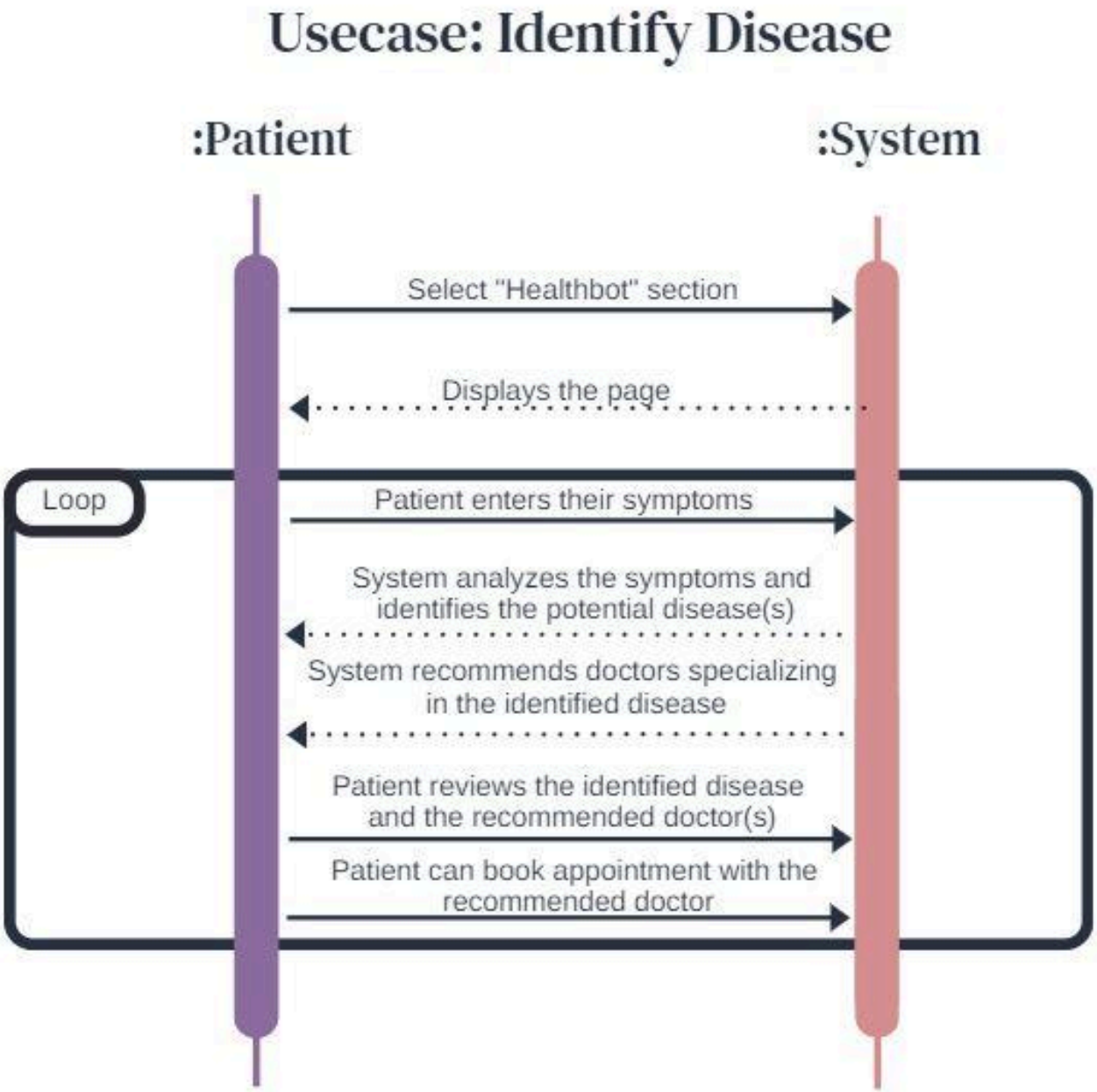


Fig 3.25 Book Appointment

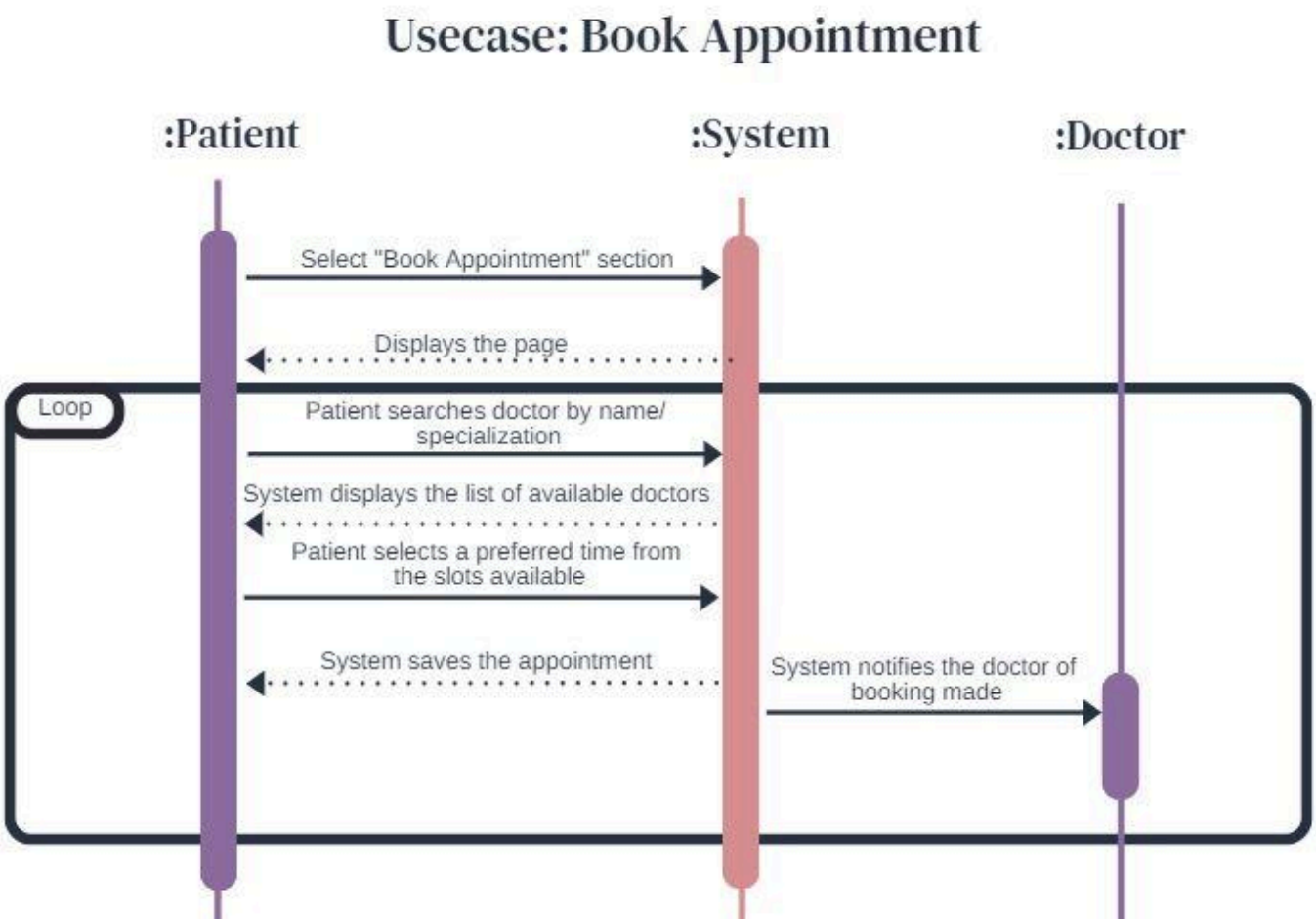


Fig 3.26 Receive Health Alerts

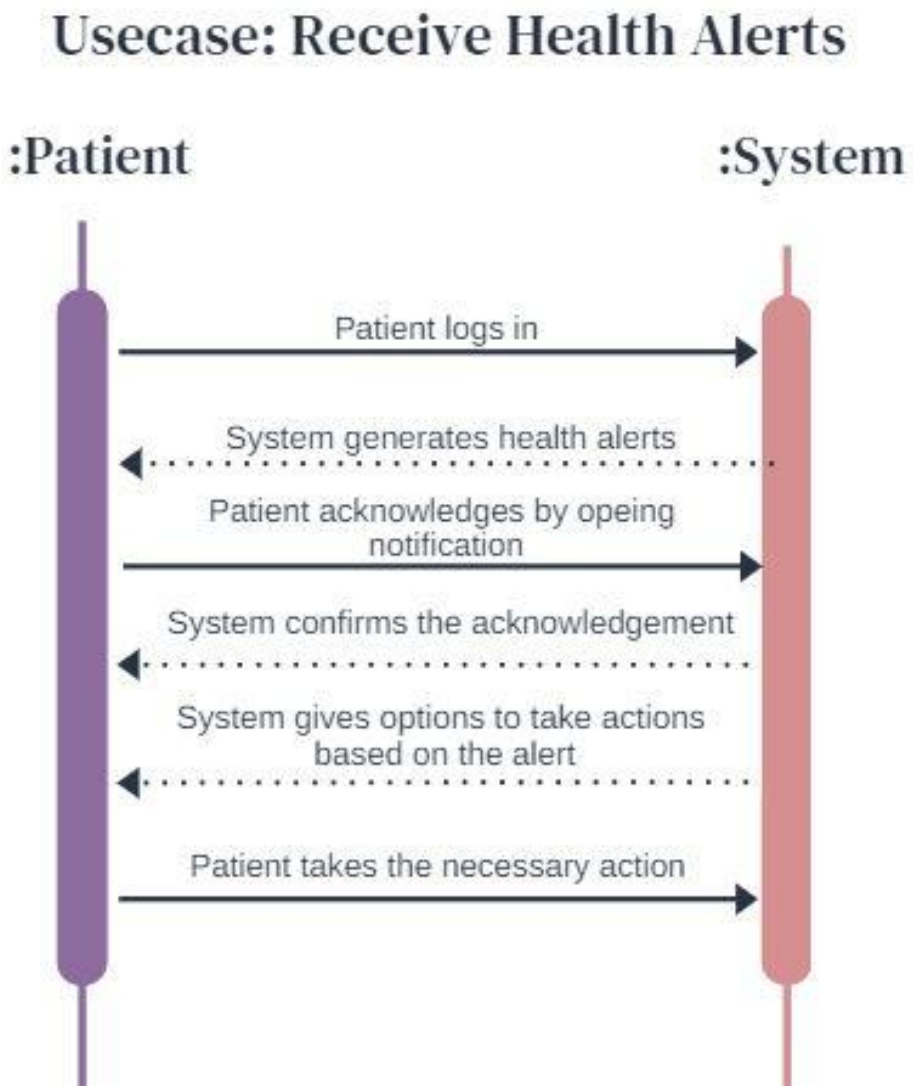


Fig 3.27 Skin Disease Detection

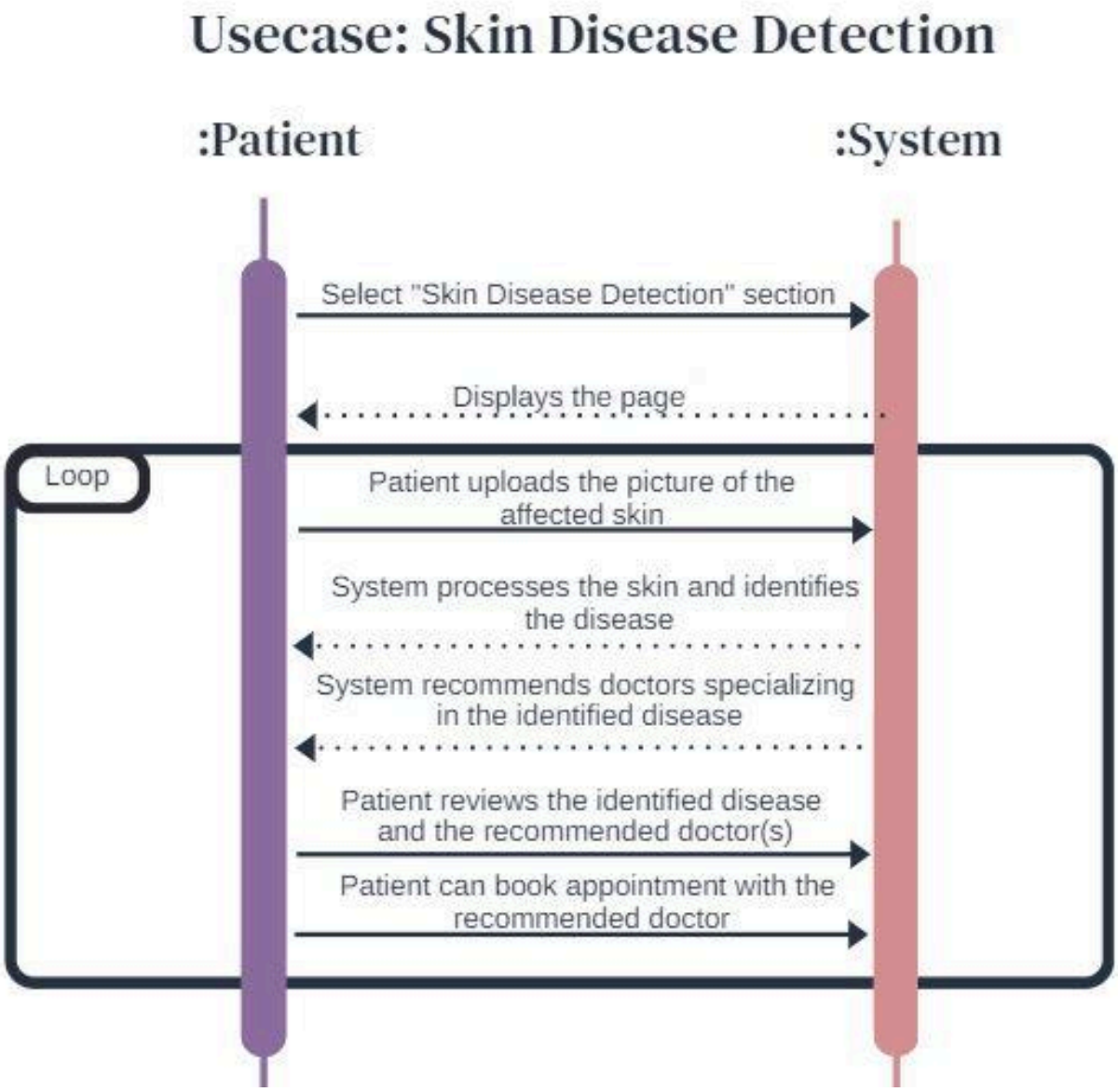


Fig 3.28 Generate Slips

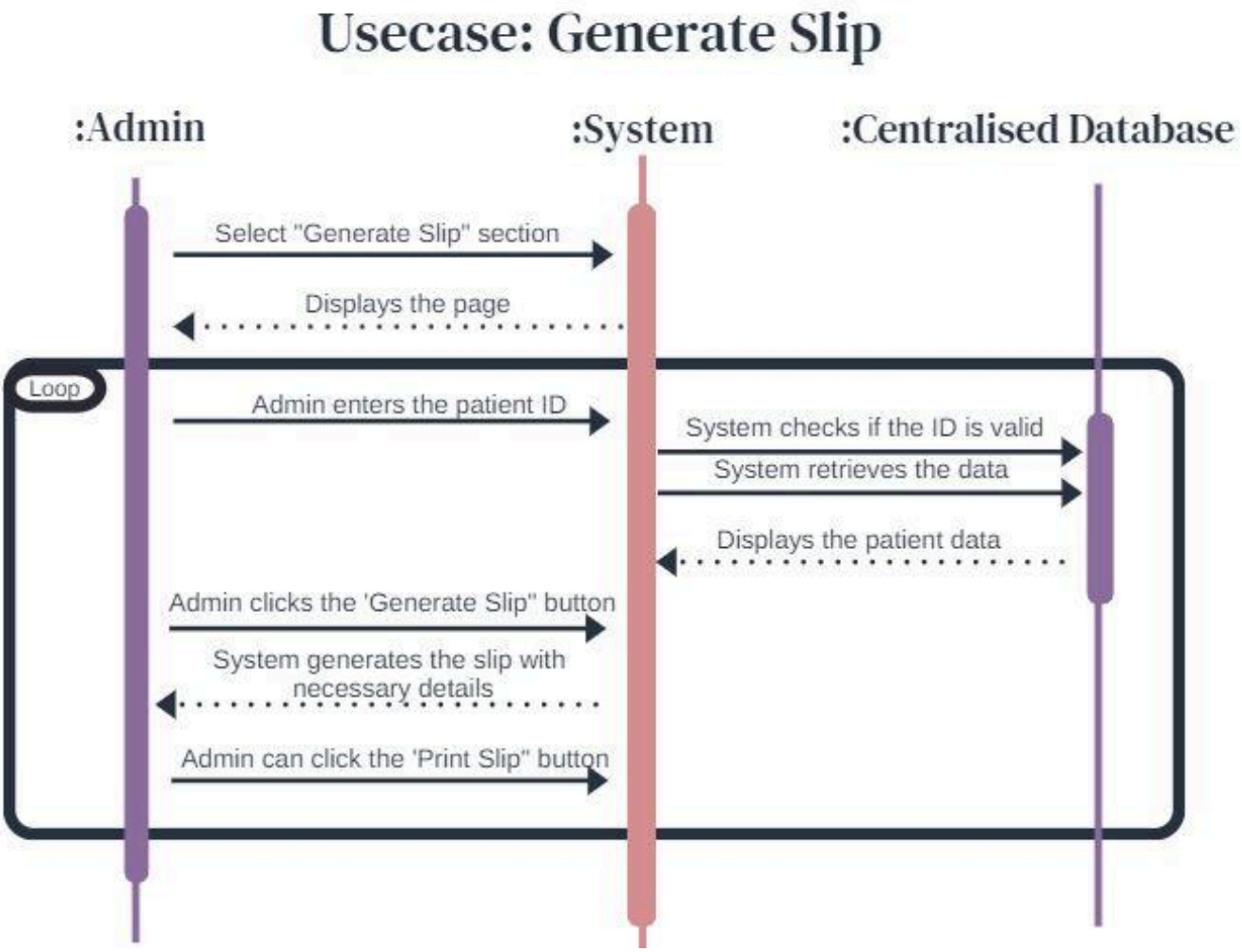
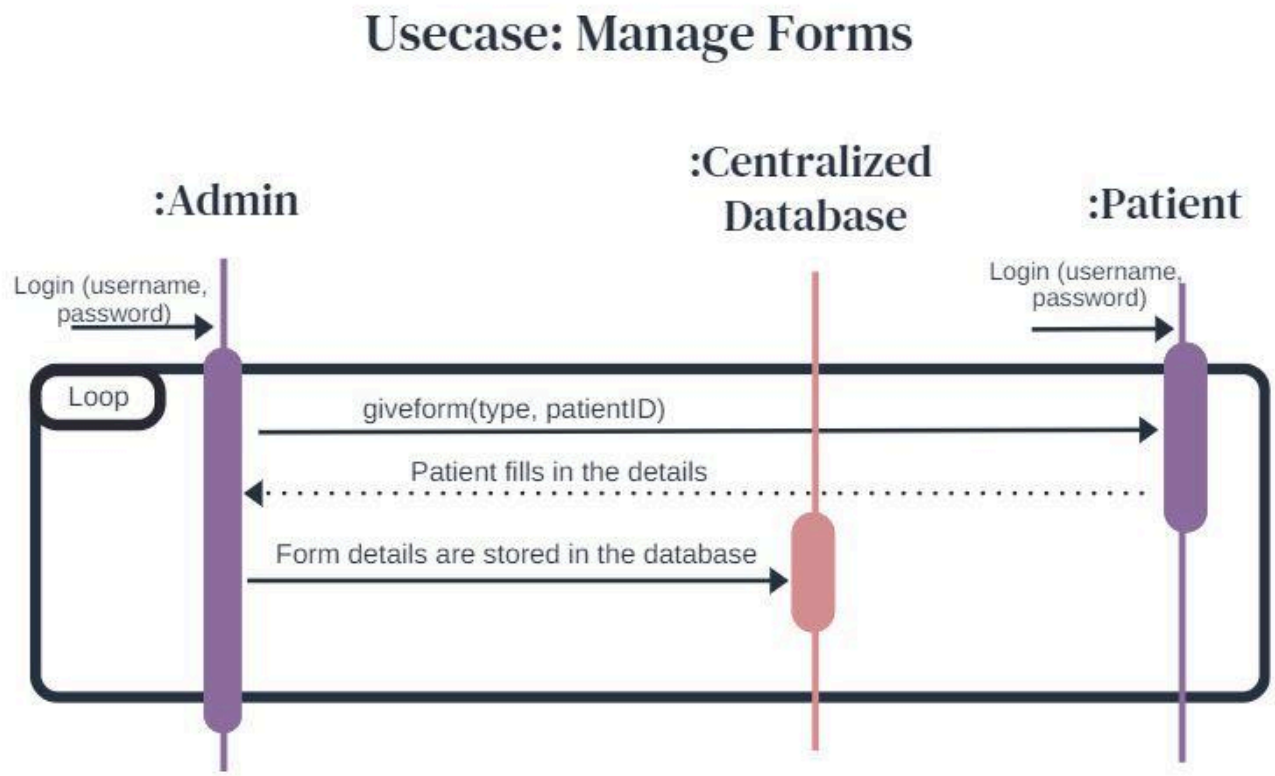


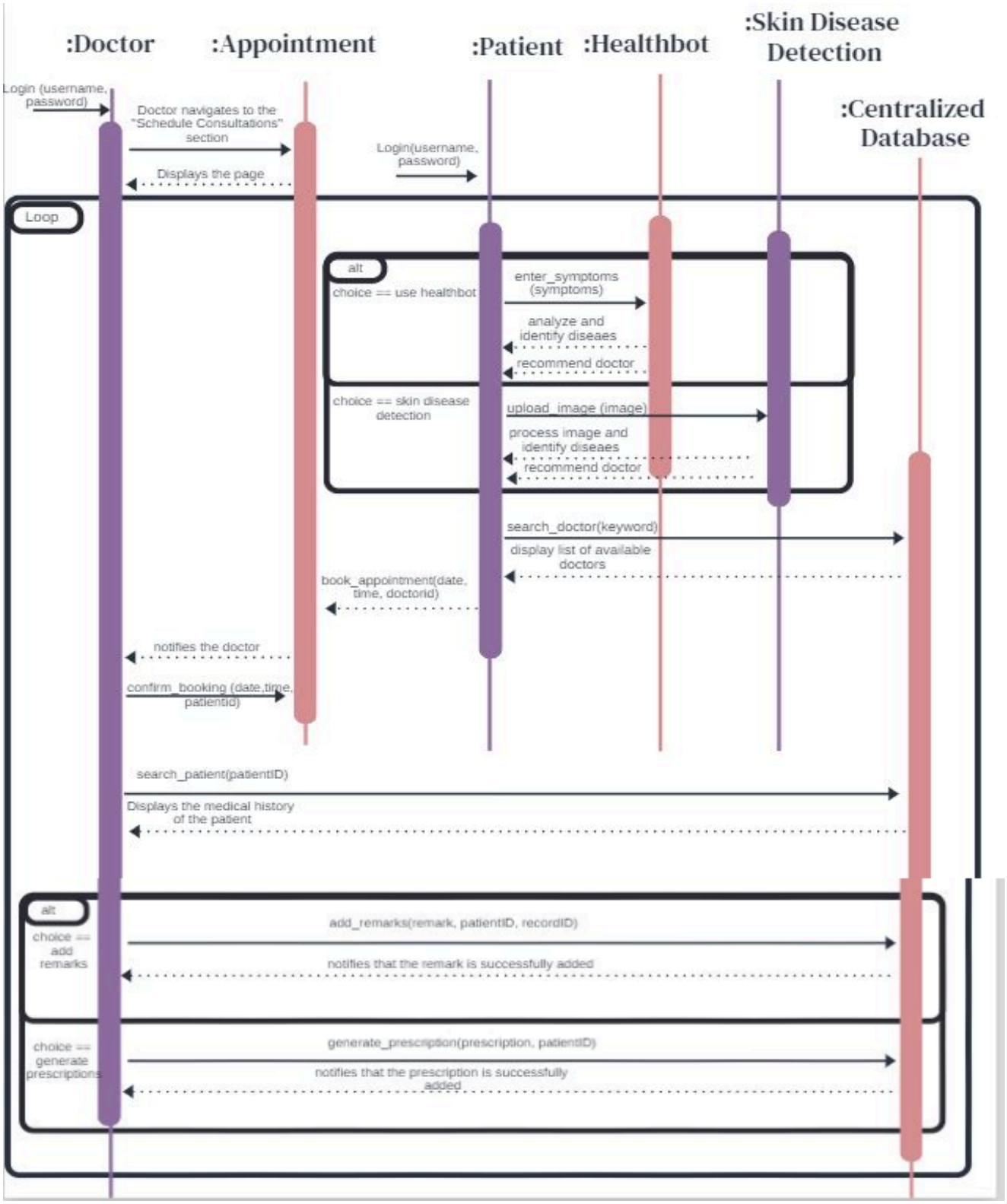
Fig 3.29 Manage Forms



3.2.4 Sequence Diagram

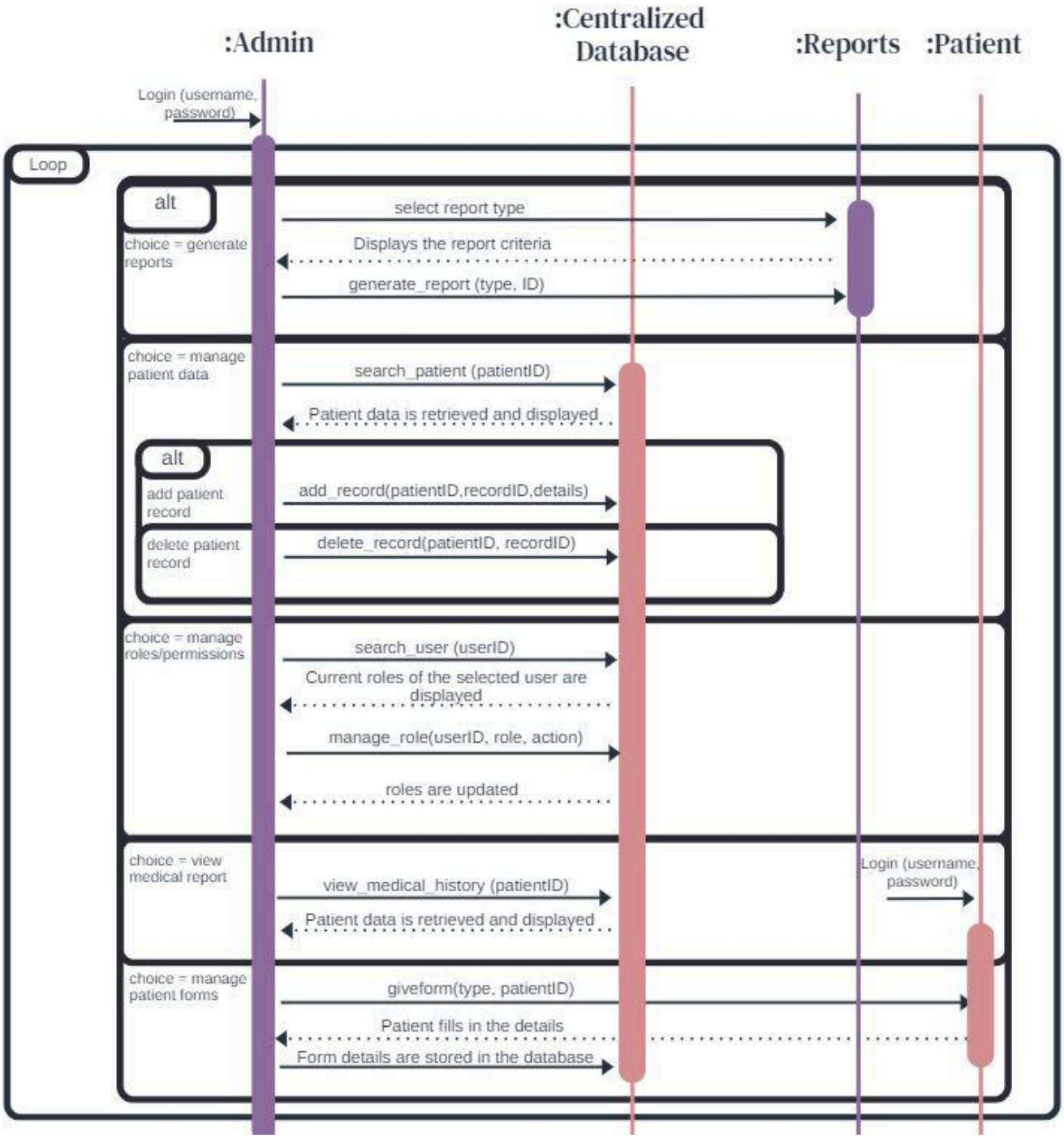
Use cases: Schedule consultation, book appointment, Identify Disease, Skin Disease detection, add remarks, generate prescriptions

Fig 3.30 Sequence Diagram 1



Use cases : Manage roles/permissions, Generate reports, manage patient data, view medical report

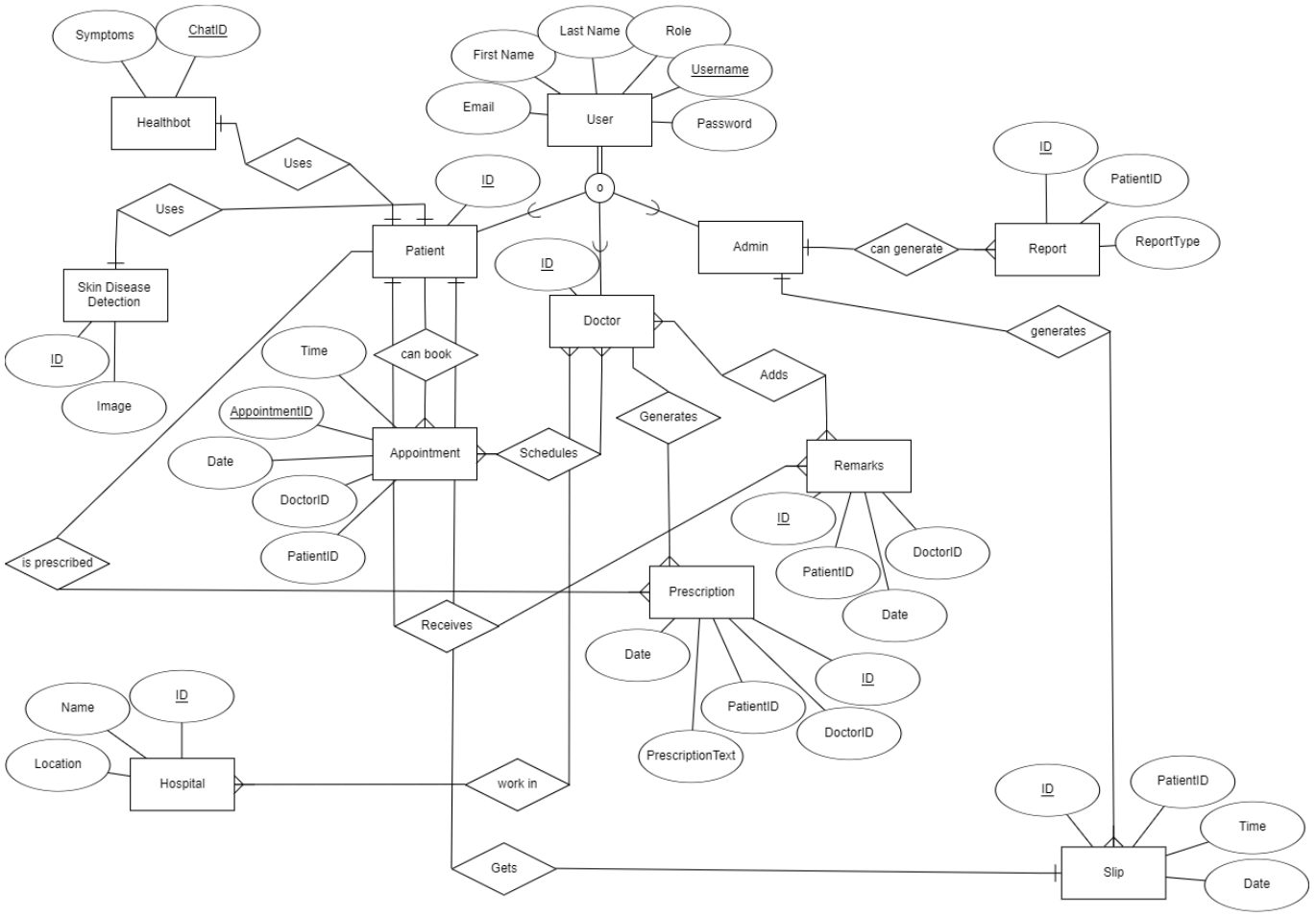
Fig 3.31 Sequence Diagram 2



3.2.5 ERD

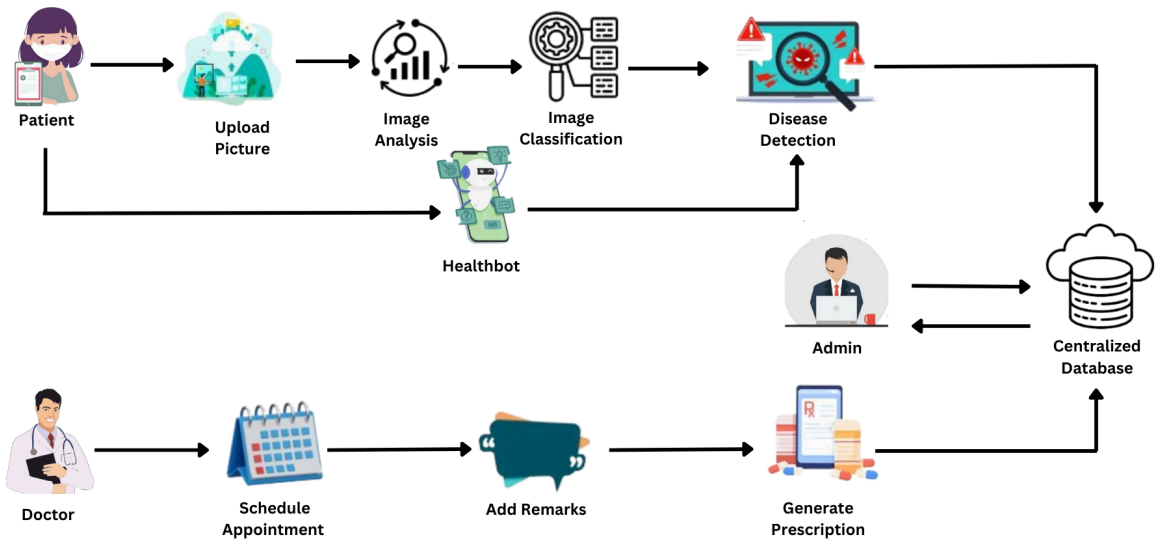
This ERD has 12 entities namely : User, Patient, Doctor, Admin, Report, Skin Disease Detection, Report, Appointment, Remarks, Hospital, Slip and Prescription. Doctor, Patient and Admin are the subclasses of the User entity and are overlapping as the doctor and admin can also be the patient. Each of the entities has its own attributes and also a key attribute for unique identity.

Fig 3.32 ERD



3.2.6 Work Flow Diagram

Fig 3.33 Work Flow Diagram



3.3 Data Design

The system's information domain consists of core entities like Patients, Doctors, Admins, Appointments, Medical Records, Roles & Permissions, Health Alerts, Reports, and the Healthbot. These entities are modeled and persist to a relational data profiles and database with the intention of proper health-related data governance. Each entity declares attributes that are mapped to fields in the database and they may also have some relationships (like one-to-many or many-to-many) with other entities.

Major Data Entities

Patient Entity:

Description: Stores patient demographics, contact information, and login credentials. Linked to other entities like Appointments, Medical Records, and Health Alerts.

Doctor Entity:

Description: Contains information about doctors, including their specialization, contact details, and login credentials. Linked to the Appointments and Medical Records entities.

Admin Entity:

Description: Manages user roles and has access to various system modules like patient data, medical records, and reports.

Appointment Entity:

Description: Represents patient appointments with doctors, storing the date, status (e.g., pending, confirmed), and relevant patient-doctor linkage.

Medical Record Entity:

Description: It contained detailed medical information like diagnosis, treatment, lab results and Doctor notes to be added.

Health Alerts Entity:

Description: Contains alerts related to patients, such as medication reminders, appointment notifications, or medical warnings.

Report Entity:

Description: Tracks administrative reports, including types such as statistical data on patient visits or hospital performance.

Healthbot Entity:

Description: Provides an AI-based diagnosis tool for patients based on symptom input, recommending doctors based on identified conditions.

Skin Disease Detection Entity:

Description: Handles skin disease detection through image analysis, identifying potential skin conditions and recommending doctors.

Data Storage Items:

Centralized Database: The main storage system that holds all the entities and relationships. The database will store structured data for:

Patient Data: Demographics, medical records, appointments, health alerts.

Doctor Data: Doctor details, consultation schedules.

Appointments and Consultations: Patient-doctor consultations.

Medical History and Prescriptions: Medical records, treatment plans.

Healthbot Interaction Data: AI-based diagnosis and recommendations.

Skin Disease Detection Data: Images and diagnostic results.

Data Storage and Organization:

A relational database like MySQL will be used to hold the structured data, which will be hosted on a cloud platform such as AWS RDS, Google Cloud SQL, or Azure Database for MySQL for scalability and reliability. Every entity will be structured into tables with their respective keys (e.g., patientID, doctorID) and foreign key relationships to link relevant data (e.g., linking appointments with patients and doctors). Cloud storage will also provide high availability, data security, and backup options. Data will be organized as follows:

Tables: Each major entity (Patient, Doctor, Appointment, Medical Record) will have a corresponding table.

Indexes: Indexes will be used for commonly queried fields like patientID, doctorID, and appointmentID to optimize query performance.

Relationships: Foreign keys will define relationships between entities (e.g., linking a patient's medical record to their ID). The system will enforce referential integrity to maintain valid relationships between records.

Bibliography

- Tariq, S., Tariq, S., & Shoukat, A. A. (2024). Centralized healthcare database for ensuring better healthcare: Are we lagging behind?. *Pakistan Journal of Medical Sciences*, *40*(3Part-II), 257.

- Kumar Jha, Y. (2023). Development of a Centralized Electronic Medical Record System–in HealthCare & Governance. *Available at SSRN 4477097*.